Hochschule Konstanz
Fakultät Elektrotechnik
und Informationstechnik

Bachelorarbeit

im Studiengang Elektrotechnik und Informationstechnik

Vergleich von ARIMA, LSTM und XGBoost zur Modellierung des Stromverbrauchs einer industriellen Anlage

Vorgelegt von

Chin-I Feng

Matrikel-Nummer: 300545

Durchgeführt an der

Hochschule Konstanz

Technik, Wirtschaft und Gestaltung

Betreuung:

Prof. Dr. Gunnar Schubert

M.Sc. Johannes Nicklaus

Konstanz, 31. Juli 2024

Kurzfassung

Diese Bachelorarbeit bewertet ARIMA-, LSTM- und XGBoost-Modelle zur Vorhersage des Stromverbrauchs der Formanlage des Unternehmens Fondium, mit Fokus auf Genauigkeit und verkürzter Trainingszeit durch Feature Engineering. Die Modelle wurden anhand der Metriken MAE, RMSE und R2 in den Validierungssätzen und Testsätzen beurteilt, um effektive Lösungen für das Energiemanagement zu identifizieren. Es zeigte sich, dass Anpassungen der Produktionsplanung basierend auf prognostizierten Stromkosten und Mustern, besonders während Spitzenpreiszeiten, die Kosten signifikant senken können.

Von den Modellen erwies sich ARIMA als am wenigsten effektiv, da es Schwierigkeiten mit nicht-linearen Schwankungen hat, während LSTM trotz längerer Trainingszeiten eine hohe Vorhersagegenauigkeit bietet. XGBoost überzeugte durch hohe Genauigkeit und Effizienz im Training, ideal für schnelle Entscheidungen auf Basis von Echtzeitdaten.

Die Analyse zeigt, dass eine optimierte Produktionsplanung die Stromkosten senken kann, demonstriert durch eine simulierte Verbesserung in der 44. Woche 2023 mit XGBoost, die eine mögliche Kostenreduktion von durchschnittlich 5,48% nahelegt. Diese Optimierung verdeutlicht die praktischen Vorteile fortschrittlicher Prognosetechniken zur Senkung der Betriebskosten und Steigerung der Energieeffizienz.

Abstract

This study evaluates ARIMA, LSTM, and XGBoost models for forecasting electricity consumption at Fondium's forming facility, focusing on enhancing model accuracy and reducing training time through feature engineering. Each model was assessed using MAE, RMSE, and R2 metrics across validation and testing phases to identify the best solutions for energy management in production. The study revealed that production planning adjustments based on predicted electricity costs and consumption patterns can significantly reduce expenses, especially during peak pricing periods.

Among the models, ARIMA was the least effective due to its inability to handle non-linear fluctuations and rapid changes, making it less suitable for dynamic industrial applications. Conversely, LSTM showed robust predictive accuracy by recognizing patterns in sequential data, although it required longer training times. XGBoost excelled in both predictive accuracy and training efficiency, making it ideal for scenarios demanding quick decisions based on real-time data.

The analysis underlined the potential for optimized production planning to lower electricity costs, as demonstrated by a simulated improvement for the 44th week of 2023 using the XGBoost model. This simulation suggested a possible average reduction of 5.48% in the original total costs for that week. This optimization highlights the practical applications of advanced forecasting in reducing operational costs and improving energy efficiency.

Inhaltsverzeichnis

Kurzfassung II
Abstract III
Abbildungsverzeichnis VI
Tabellenverzeichnis VIII
Abkürzungsverzeichnis IX
1 Einleitung 1
2 Theoretische Grundlagen 3
2.1 Modelle 3
2.1.1 ARIMA 3
2.1.2 LSTM 5
2.1.3 XGBoost 7
2.2 Korrelation 9
2.2.1 Spearman-Rangkorrelation 9
2.2.2 Autokorrelation 10
2.2.3 Partielle Autokorrelation 11
2.3 Hypothesentest 11
2.4 Modell-Evaluierung 13
2.4.1 Cross-Validation 13
2.4.2 Evaluierungsmetriken 15
2.5 Python-Pakete 16
3 Vorverarbeitung der Zeitreihendaten 19
3.1 Laden der Daten 19
3.2 Verwendung der Datums- und Zeitindizes 21
3.3 Datenbereinigung 22
3.4 Feature Engineering 23
3.5 Datenaufteilung 27
4 Entwicklung der Modelle 29
4.1 ARIMA 29
4.1.1 Auswahl der Parameter 29
4.1.2 Schritte zur Modellierung 31
4.1.3 ARIMA-Prognosen zum Stromverbrauch 32

4.2 LSTM 33
4.2.1 Auswahl der Hyperparameter 33
4.2.2 Schritte zur Modellierung 36
4.2.3 LSTM-Prognosen zum Stromverbrauch 36
4.3 XGBoost 38
4.3.1 Auswahl der Hyperparameter 38
4.3.2 Schritte zur Modellierung 39
4.3.3 XGBoost-Prognosen zum Stromverbrauch 40
5 Auswertung der Modelle 42
5.1 Leistungsvergleich der Vorhersagemodelle 42
5.2 Effizienzvergleich der Vorhersagemodelle 43
5.3 Zusammenfassung der Vergleichsergebnisse 43
6 Anwendung 45
7 Fazit und Ausblick 50
Literaturverzeichnis X
Eidesstattliche Erklärung XIII
Anhang XIV

Abbildungsverzeichnis

Abb. 2.1: LSTM-Architektur 6
Abb. 2.2: Deep-Learning Netzwerk 7
Abb. 2.3: XGBoost-Architektur 8
Abb. 2.4: Wahrscheinlichkeitsverteilung mit 0,05 Signifikanzschwelle 12
Abb. 2.5: Einmalige Modellbewertung 13
Abb. 2.6: TimeSeriesSplit-Methode der Cross-Validation 14
Abb. 3.1: Datenimport mit Pandas 19
Abb. 3.2: Vorschau der Daten 20
Abb. 3.3: Übersicht der DataFrame-Struktur 20
Abb. 3.4: Zeitstempel-Konvertierungsprozess 21
Abb. 3.5: Überprüfung der Zeitstempel-Indizierung 21
Abb. 3.6: Histogramm der Stromverbrauchsverteilung 22
Abb. 3.7: Entfernen von Ausreißern und Nullwerten im Stromverbrauch 23
Abb. 3.8: Spearman-Korrelationsmatrix der Produktionsdaten mit p-Werten 24
Abb. 3.9: Vergleich des täglichen Stromverbrauchs: Feiertage, Arbeitstage und Produktionsausfälle 25
Abb. 3.10: Boxplots des stündlichen Stromverbrauchs 25
Abb. 3.11: Violinplot des Stromverbrauchs nach Schicht 26
Abb. 3.12: Übersicht der Features und Zielvariablen im Modell 27
Abb. 3.13: Aufteilung der Zeitreihendaten in Trainings- und Testdaten 27
Abb. 3.14: Visualisierung der Stromverbrauchsverteilung 28
Abb. 4.1: Stationaritätstest der Zeitreihendaten mittels ADF-Test 29
Abb. 4.2: ADF-Test Ergebnisse zur Bestimmung der Stationarität 30
Abb. 4.3: ACF- und PACF-Diagramme zur Ermittlung von p und q 30
Abb. 4.4: Programmcode zur Implementierung eines ARIMA-Modells 31
Abb. 4.5: Darstellung der Stromverbrauchsprognose durch ARIMA 32
Abb. 4.6: Residualplot der ARIMA-Vorhersage für den Stromverbrauch 32
Abb. 4.7: LSTM-Leistung bei unterschiedlicher Anzahl an Neuronen. Im weiteren Verlauf wurden 16 Neuronen verwendet. 33
Abb. 4.8: LSTM-Leistung bei verschiedenen Lernraten. Im weiteren Verlauf wurde eine Lernrate von 0.07 verwendet. 34
Abb. 4.9: LSTM-Leistung bei verschiedenen Batch-Größen. Im weiteren Verlauf wurde eine Batch-Größe von 32 verwendet. 35
Abb. 4.10: LSTM-Leistung bei verschiedenen Lags. Im weiteren Verlauf wurde ein maximaler Lag von 5 verwendet. 36
Abb. 4.11: Darstellung der Stromverbrauchsprognose durch LSTM 37
Abb. 4.12: Residualplot der LSTM-Vorhersage für den Stromverbrauch 37
Abb. 4.13: XGBoost-Leistung bei verschiedenen Lernraten. Im weiteren Verlauf wurde eine Lernrate von 0.3 verwendet. 38

Abb. 4.14: XGBoost-Leistung bei verschiedenen Estimatoren. Im weiteren Verlauf wurden 100 Estimatoren verwendet. 39
Abb. 4.15: Programmcode zur Implementierung eines XGBoost-Modells 40
Abb. 4.16: Darstellung der Stromverbrauchsprognose durch XGBoost 40
Abb. 4.17: Residualplot der XGBoost-Vorhersage für den Stromverbrauch 41
Abb. 6.1: Strompreis- und Verbrauchsdynamik, 03. November 2023 45
Abb. 6.2: Optimierung der Stromkosten durch Anpassung der Verbrauchsdynamik, 03. November 2023 47
Abb. 6.3: Eingesparte Stromkosten und Prozentanteile der 44. Woche 2023. Der erste Tag (30.10.2023) ist ein Montag. 48

Tabellenverzeichnis

Tab. 2.1: Klassifizierung der Modelle nach Typ 3
Tab. 5.1: Leistungsvergleich von ARIMA, LSTM und XGBoost 42
Tab. 5.2: Effizienzvergleich von ARIMA, LSTM und XGBoost 43
Tab. 6.1: Spearman-Korrelationen zwischen optimierter und nicht optimierter Produktionsplanung und Strompreisen mit p-Werten 48

Abkürzungsverzeichnis

ACF	Autokorrelationsfunktion
ADF-Test	Augmented Dickey-Fuller-Test
ARIMA	Autoregressive Integrated Moving Average
CSV	Comma-Separated-Values
CUDA	Compute Unified Device Architecture
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DL	Deep Learning
KI	Künstliche Intelligenz
kWh	Kilowattstunde
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
PACF	Partielle Autokorrelationsfunktion
R2	R-Squared
RMSE	Root Mean Square Error
RNN	Rekurrentes neuronales Netzwerk
XGBoost	Extreme Gradient Boosting

1 Einleitung

Die herkömmliche Datenanalyse ist heutzutage wegen der immer größeren Datenmengen und der hohen Anzahl der Produktionsparameter keine triviale Aufgabe für viele Fertigungsunternehmen [1]. Außerdem erschwert die Komplexität der Daten es, Potenziale zur Energieeinsparung aus dem Produktionsprozess zu identifizieren sowie den eigenen künftigen Stromverbrauch in Produktionsprozessen zu modellieren und präzise zu prognostizieren. Diese Lücke in der Energieverwaltung führt zu einer verpassten Gelegenheit, die Stromkosten in der Fertigung zu senken [2].

Als Pioniere der Zeitreihenanalyse können Statistiker George Box und Gwilym Jenkins genannt werden, die insbesondere zu dem Bereich der Benchmark der Vorhersagemodelle mit Zeitreihen beitrugen. Des Weiteren sind sie die Begründer der Box-Jenkins-Methode, mit der man die Modelle für einen bestimmten Zeitreihendatensatz identifizieren und überprüfen kann [3]. Andere untersuchten auch die Möglichkeiten dieser Methode, die Vorhersagemodelle z.B. im Bereich Geschäft und Wirtschaft zu entwickeln [4]. Das Problem besteht darin, dass noch kein Modell für die industrielle Formanlage im Bereich Eisenguss zur Verfügung steht, mit dem man eine Vorhersage für den künftigen Stromverbrauch treffen kann.

Das Hauptziel dieser Bachelorarbeit ist es daher, die Produktionsdaten bzw. Produktionsparameter der Firma Fondium aus Singen (Hersteller für Eisengussteile) jeweils mithilfe der Nutzung von Statistik, Deep Learning (DL) und Machine Learning (ML) zu analysieren, um den Stromverbrauch einer großindustriellen Formanlage so genau wie möglich zu prognostizieren. Durch diese Modellierung der Formanlage soll das Datenmuster des Produktionsverfahrens erkannt werden, mit dem man Vorhersagen über künftige Energiebedarfe treffen kann. Mithilfe der Vorhersage lassen sich schlussendlich durch effektive Analysemethoden Wege finden, wie die Produktion auf Zeiten mit niedrigeren Stromkosten verlagert werden kann. Ziel der Arbeit wird es auch sein, die oben genannten Modelle zu validieren, um die maximale Vorhersagegenauigkeit bei minimaler Trainingsdauer bewerten zu können. Daraus ergeben sich die folgenden Forschungsfragen:

(1) Wie können Produktionsdaten effektiv als Modell-Eingabe ausgewählt werden, um die Leistung der Vorhersagemodelle zu verbessern?

(2) Welches Modell bietet die maximale Vorhersagegenauigkeit bei minimaler Trainingsdauer?

(3) Inwiefern beeinflusst die Optimierung des Stromverbrauchs durch präzise Modellierung und Prognose die Stromkosten?

Zuerst werden die verfügbaren Rohdaten bereinigt und zu standardisierten Zeitreihendaten verarbeitet, um eine Integration in die Analyseumgebung zu ermöglichen [5]. Anschließend werden aussagekräftige Merkmale aus den Daten extrahiert und zur Implementierung der drei Modelle verwendet. Auf die Modellierung und das Modelltraining folgt eine statistische Methode zur Bewertung der Leistung der Modelle mittels Cross-Validation. Im Anschluss der Modellanpassung besteht der letzte Teil der Arbeit in der Vorhersage des Stromverbrauchs, welcher die Entscheidung für die Reduzierung des Stromverbrauchs unterstützt, damit man die Produktion in Phasen mit günstigeren Strompreisen verschieben kann.

Im folgenden Verlauf der Arbeit wird die Entwicklung dieser Vorhersagemodelle beschrieben, wobei sowohl die Minimierung der Trainingsdauer als auch die Maximierung der Vorhersagegenauigkeit untersucht werden. Beginnend mit dem **Kapitel 2** werden die theoretischen Grundlagen, die in dieser Arbeit angewandten Methoden, Algorithmen und Hypothesentests erläutert. Darauf folgt in **Kapitel 3** die Voranalyse der Produktionsdaten sowie die Methodik zur Reduktion von Parametern, um die Mustererkennung zu vereinfachen.

In **Kapitel 4** werden die Entwicklung der Modellierung der Formanlage und deren Implementierung in Python sowie die einzelnen Schritte wie Modelltraining und Modellanpassung vorgestellt. Anschließend wird in **Kapitel 5** auf die Ergebnisse der Modellleistung (die Vorhersagegenauigkeit und Trainingsdauer) eingegangen und in **Kapitel 6** auf die mögliche zukünftige Anwendung des Modells in der Produktion zur Reduzierung der Stromkosten. Am Ende erfolgt in **Kapitel 7** das Fazit und der Ausblick für zukünftige Verbesserungen des Vorhersagemodells.

2 Theoretische Grundlagen

In diesem Kapitel werden für ein tieferes Verständnis die verwendeten Modelle dieser Arbeit erläutert. Des Weiteren werden die relevanten Korrelationsanalysen vorgestellt. Anschließend werden die Metriken für die Bewertung der Vorhersagemodelle für eine große Datenmenge mit Zeitreihen festgelegt, welche in dieser Arbeit eingesetzt werden. Schlussendlich werden auch die Python-Pakete vorgestellt, die zur Unterstützung dieser Funktionen verwendet werden.

2.1 Modelle

Um künftige Ereignisse wie den Stromverbrauch vorhersagen zu können, werden die bisher gesammelten Daten zur Analyse historischer Ereignisse verwendet. Dazu müssen die wesentlichen Aspekte des Sachverhalts mathematisch beschrieben werden, um das Modell effektiv nachbilden zu können. Dieses Modell ermöglicht es, durch die Integration von Algorithmen, Muster und Regeln bei gegebenen Problemstellungen selbstständig zu erkennen.

Basierend auf den jeweiligen Anforderungen der Anwendung werden angemessene Modelle ausgewählt. Für den speziellen Kontext der Zeitreihenprognosen kommen folgende Modelle zum Einsatz, die in den nächsten Unterkapiteln konkret vorgestellt werden:

Tab. 2.1: Klassifizierung der Modelle nach Typ

Modellart	**Modell**
Statistik	Autoregressive Integrated Moving Average (ARIMA)
DL	Long Short-Term Memory (LSTM)
ML	Extreme Gradient Boosting (XGBoost)

2.1.1 ARIMA

Das ARIMA-Modell ist ein stochastischer Modellierungsansatz, der sich aus dem AR(p)-Modell, dem I(d)-Modell und dem MA(q)-Modell zusammensetzt. Es wurde von George Box und Gwilym Jenkins als eine statistische Methode zur Analyse und Erstellung eines Prognosemodells entworfen und wird daher auch Box-Jenkins-Methode genannt [6].

- **AR(p)**: AR bezeichnet ein selbstregressives Modell. Das bedeutet, dass der aktuelle Wert eines Zeitpunkts (z_t) eine Regression der Werte mehrerer vergangener Zeitpunkte ist. Wenn das Modell von den letzten p historischen

Werten abhängt, wird die Ordnung als p bezeichnet und das Modell als AR(p)-Modell notiert [3]. Die Formel lautet:

$$z_t = \phi_1 z_{t-1} + \phi_2 z_{t-2} + \cdots + \phi_p z_{t-p} + \epsilon_t \tag{2.1}$$

,mit

$z_t =$ Der Beobachtungswert zum Zeitpunkt t

$\phi_1, \phi_2, \ldots, \phi_p =$ Die Koeffizienten der Lag von z bis zum p-ten Lag

$\epsilon_t =$ Der Fehlerterm der aktuellen Beobachtung

- **I(d)**: I bezeichnet ein integriertes Modell, welches eine Differenzierung der Zeitreihe durchführt. Da die Zeitreihenanalyse stationäre Daten erfordert, müssen nicht-stationäre Reihen durch bestimmte Methoden in stationäre Reihen umgewandelt werden, wobei häufig die Differenzierung verwendet wird [3]. Die Formel lautet:

$$\nabla^d z_t = (1 - L)^d z_t \tag{2.2}$$

,mit

$\nabla^d z_t =$ Die d-te Differenz von z_t

$L =$ Der Lag-Operator zur rückwärtigen Verschiebung einer Zeitreihe

- **MA(q)**: MA bezeichnet ein gleitendes Durchschnittsmodell. Das bedeutet, dass der aktuelle Wert eines Zeitpunkts eine Regression der Vorhersagefehler mehrerer vergangener Zeitpunkte ist. Wenn die Reihe von den letzten q historischen Vorhersagefehlern abhängt, wird die Ordnung als q bezeichnet und das Modell als MA(q)-Modell dargestellt [3]. Die Formel lautet:

$$z_t = \epsilon_t - \theta_1 \epsilon_{t-1} - \theta_2 \epsilon_{t-2} - \cdots - \theta_q \epsilon_{t-q} \tag{2.3}$$

,mit

$z_t =$ Der Beobachtungswert zum Zeitpunkt t

$\theta_1, \theta_2, \ldots, \theta_q =$ Die Koeffizienten der Fehlerterme

$\epsilon_t, \epsilon_{t-1}, \ldots, \epsilon_{t-q} =$ Die Fehlerterme

In der allgemeinen Form des ARIMA-Modells wird z_t^*, das Ergebnis der d-ten Differenzierung der ursprünglichen Zeitreihe z_t als Funktion der p-vorgeschriebenen AR-Parameter und der q-vorgeschriebenen MA-Parameter betrachtet. Die Variablen ϵ und z_t^* sind integraler Bestandteil der Berechnungen in den AR- und MA-Modellen. Der vordere Teil der Formel beinhaltet AR-Parameter, die auf z_t^* angewendet werden, während der hintere Teil die MA-Parameter enthält, die die kurzfristigen Zufallsschwankungen beschreiben [3]. Die allgemeine Form des ARIMA-Modells lautet:

$$
\begin{aligned}
z_t^* &= \overbrace{\nabla^d z_t}^{\text{I(d)-Modell}} \\
z_t^* &= \underbrace{\sum_{i=1}^{p} \phi_i z_{t-i}^* + \epsilon_t}_{\text{AR(p)-Modell}} - \underbrace{\sum_{i=1}^{q} \theta_i \epsilon_{t-i}}_{\text{MA(q)-Modell}}
\end{aligned}
\tag{2.4}
$$

In der klassischen Anwendung liefern ARIMA-Modelle deterministische Vorhersagen, sobald sie einmal trainiert wurden. Das bedeutet, dass die Vorhersagen mit identischen Parametern (p, d, q) für denselben Datensatz immer gleich sein werden. Sie generieren punktgenaue Prognosen für zukünftige Werte, basierend auf historischen Zeitreihendaten und statistischen Beziehungen, ohne Wahrscheinlichkeitsverteilungen möglicher Ergebnisse anzugeben.

2.1.2 LSTM

LSTM ist eine weiterentwickelte Form des rekurrenten neuronalen Netzwerks (RNN), das speziell für DL-Anwendungen in Zeitreihendaten entwickelt wurde. LSTMs können sich über längere Zeiträume an frühere Informationen erinnern und besser langfristige Muster in sequenziellen Daten erkennen. D.h. im Gegensatz zu herkömmlichen RNNs haben LSTMs ein erstaunliches Gedächtnis. Herkömmliche Netzwerke haben dagegen häufig Schwierigkeiten, diese Langzeitabhängigkeiten zu lernen, weil sie keinen Gating-Mechanismus besitzen. Durch die Implementierung eines Gating-Mechanismus, bestehend aus einem Forget-Gate, Input-Gate und Output-Gate, adressiert LSTM dieses Phänomen effektiv [7].

Jede Zelle innerhalb eines LSTM-Netzwerks beinhaltet einen Zellzustand, welcher als Informationsträger fungiert und diese Daten weiterleitet. Der Gating-Mechanismus reguliert die Aktualisierung und das Verwerfen dieses Zustands und kontrolliert somit die Informationsaufnahme und -ausgabe. Dieser Zustand wird kontinuierlich durch den Gating-Mechanismus moduliert, indem irrelevante Inhalte gezielt vergessen und neue Eingaben basierend auf den Gating-Aktivierungen integriert werden. Die Kernfunktionalität von LSTM liegt in der strukturierten Kontrolle dieses internen Informationsflusses zur effizienten Modellierung temporaler Dynamiken [7].

Im Gating-Mechanismus wird das Forget-Gate durch eine Sigmoid-Funktion kontrolliert. Die Eingabe umfasst den gegenwärtigen Wert (X_t) sowie den früheren versteckten Zustand (h_{t-1}). Der Ausgabewert des Forget-Gates liegt zwischen 0 und 1 und bestimmt, wie viel vom vorherigen Zellzustand (C_{t-1}) erhalten oder vergessen wird. Die Kontrolle des Input-Gates erfolgt auch mithilfe einer Sigmoid-Funktion, die sowohl den aktuellen Eingabewert als auch den vorherigen versteckten Zustand als

Eingang verwendet. Der Ausgabewert des Input-Gates zwischen 0 und 1 regelt die Integration neuer Informationen in den Zellzustand. Das Output-Gate, ebenfalls reguliert durch eine Sigmoid-Funktion, entscheidet darüber, welcher Anteil des aktuellen Zellzustands (C_t) als der nächste versteckte Zustand (h_t) weitergegeben wird. Diese Entscheidung beeinflusst, welche Informationen letztendlich übertragen werden. Durch diese Funktion des LSTM können Zellzustände (C_t) über sehr viel längere Zeiträume beibehalten werden [7]. Eine Visualisierung des Gating-Mechanismus ist in Abb. 2.1 dargestellt.

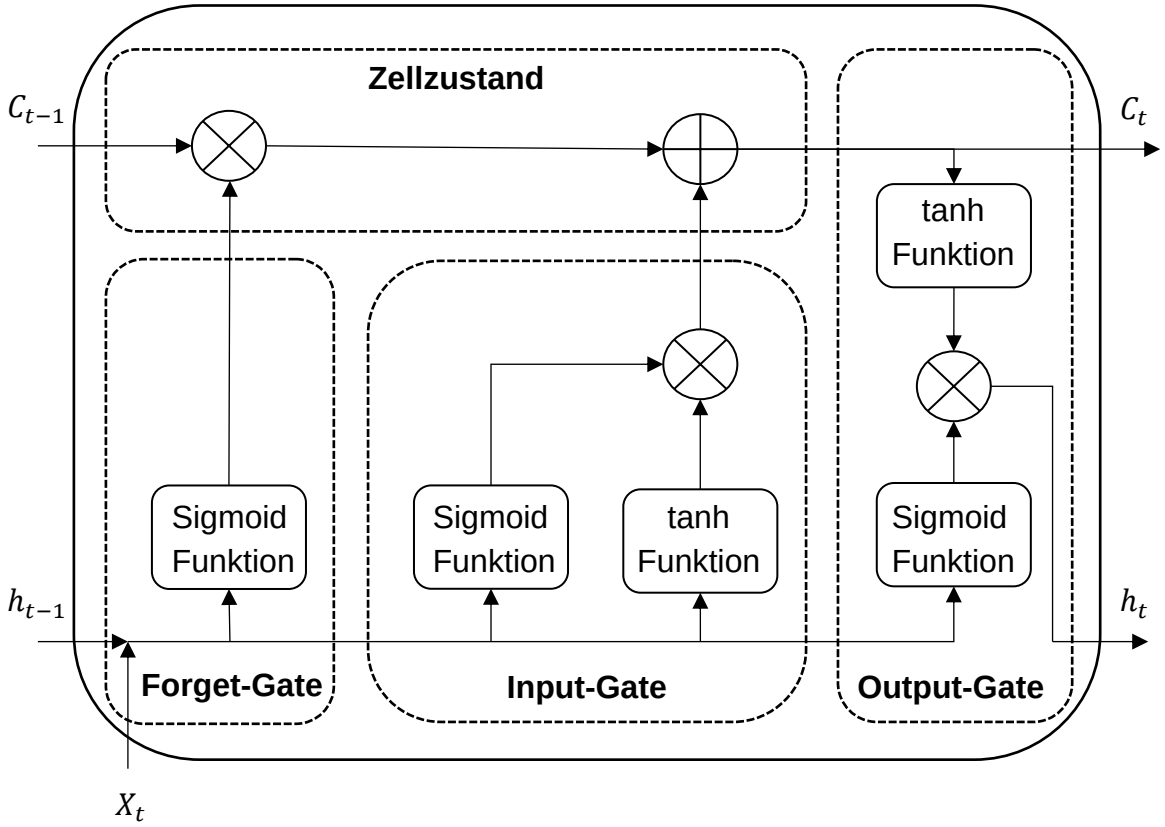

Abb. 2.1: LSTM-Architektur

Wenn diese Zellen miteinander kombiniert werden, entsteht ein robustes DL-Netzwerk. Dieses Netzwerk ist fähig, komplexe Muster in Zeitreihendaten zu erkennen und zu interpretieren, indem es nicht nur aktuelle Eingaben, sondern auch frühere Zustände berücksichtigt. Der Input-Layer nimmt die initialen Daten auf, wie z.B. Zeitreihenwerte. Diese Daten werden dann durch einen oder mehrere Hidden-Layers verarbeitet, die die Komplexität und die Muster in den Daten durch verschiedenartige Transformationen und Verbindungen erkennen. In dieser Arbeit nutzen die Hidden-Layers ausschließlich LSTM-Zellen, um relevante Merkmale und Zusammenhänge in den Zeitreihendaten zu identifizieren und weiterzuleiten. Dies ist besonders vorteilhaft für komplexe Prognoseaufgaben, bei denen Vorhersagen auf der Grundlage von

historischen Datenmustern getroffen werden müssen. Schließlich interpretiert der Output-Layer die von den Hidden-Layers aufbereiteten Informationen und generiert die Prognose (siehe Abb. 2.2) [8].

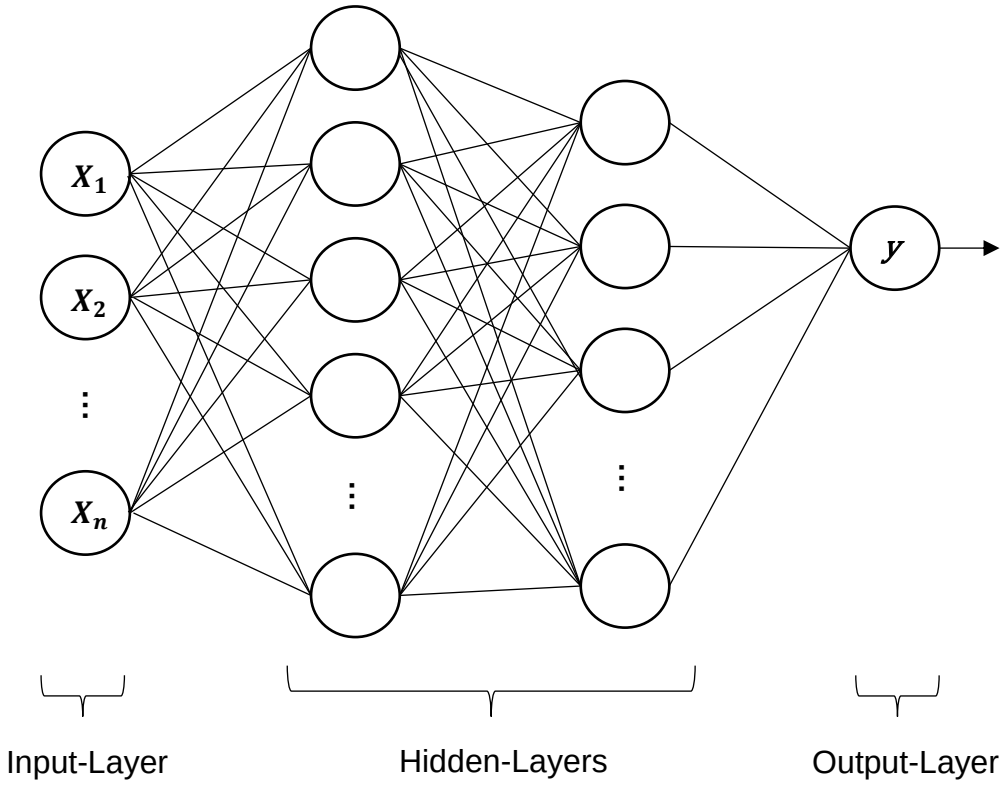

Abb. 2.2: Deep-Learning Netzwerk

Im Gegensatz zum ARIMA-Modell, welches deterministische Vorhersagen liefert, kann LSTM für probabilistische Vorhersagen konfiguriert werden. Aufgrund seiner rekurrenten Natur ist es in der Lage, eine Verteilung möglicher zukünftiger Werte zu prognostizieren, wodurch Unsicherheiten und Wahrscheinlichkeiten verschiedener zukünftiger Ereignisse abgebildet werden können [9].

2.1.3 XGBoost

XGBoost ist eine Ensemble-Methode (ein überwachter ML-Algorithmus), die viele schwache Vorhersagemodelle (oft Entscheidungsbäume) kombiniert, um robuste Vorhersagen zu erzielen. Das Modell fügt jeweils nur einen neuen Baum hinzu, anstatt alle Bäume auf einmal zu erstellen [10]. Die allgemeine Form des XGBoost-Modells lautet:

$$y_i = \sum_{k=1}^{n} f_k(x_i) \tag{2.5}$$

,mit

$n =$ Die Anzahl der zu erstellenden Bäume

$f_k(x_i) =$ Die Funktion der Eingabe im k-ten Entscheidungsbaum

$y_i =$ Der vorhergesagte Wert

Das Training der Entscheidungsbäume erfolgt durch das Gradientenabstiegsverfahren. In jedem Boosting-Schritt wird der negative Gradient der Verlustfunktion bezüglich der aktuellen Prognosen berechnet und als Zielvariable für den neuen Baum verwendet, der die Residuen der bisherigen Vorhersagen approximiert (siehe Abb. 2.3). Durch die schrittweise Minimierung der Verlustfunktion nähert sich das Modell iterativ einem Optimum an. Zudem verwendet XGBoost während des Trainings spaltenbasiertes Subsampling der Merkmale, eine Methode, bei der ausgewählte Merkmale statt ganzer Datensätze genutzt werden, ähnlich dem Random-Forest-Verfahren. Dies dient dazu, die Korrelation zwischen den Bäumen zu reduzieren und die Robustheit des Modells zu erhöhen [10].

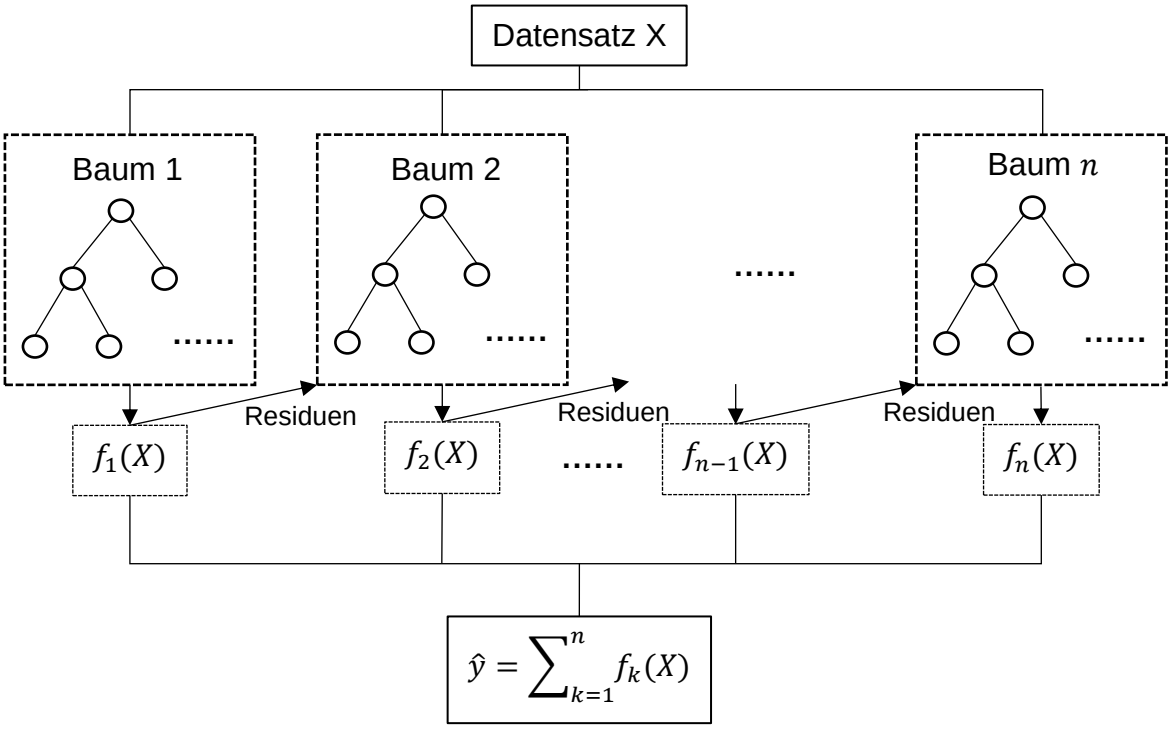

Abb. 2.3: XGBoost-Architektur

XGBoost zeichnet sich durch verschiedene algorithmische Optimierungen für effizientes Training aus, einschließlich der Vorberechnung von Statistiken und der Behandlung fehlender Werte. Um den Speicherbedarf zu reduzieren, nutzt XGBoost

das Out-of-Core-Verfahren zum blockweisen Laden der Trainingsdaten. Darüber hinaus unterstützt der Algorithmus verteiltes Training auf Cluster-Architekturen sowie Multi-Threading auf einzelnen Maschinen, um die verfügbaren Ressourcen optimal auszulasten und die Trainingsdauer des Modells zu verkürzen [10].

In der praktischen Anwendung spielt das Hyperparameter-Tuning eine wichtige Rolle für die Leistung von XGBoost. Zu den entscheidenden Parametern zählen unter anderem die Lernrate (steuert das Lerntempo), die Anzahl der Estimatoren (Entscheidungsbäume), die maximale Baumtiefe (regelt die Komplexität der Bäume) und die Mindestanzahl der Beobachtungen pro Blatt (verhindert Overfitting durch Mindestgröße der Blätter) [11] [12]. In der Standardanwendung produziert XGBoost wie das ARIMA-Modell deterministische Vorhersagen. Die Entscheidungsbäume, die während des Trainings gebaut werden, verändern sich nicht, wenn das Modell mit identischen Daten und Parametern erneut angewendet wird.

2.2 Korrelation

Bei der Modellierung ist es entscheidend, die effektiven Eingabeparameter für das Modell zu identifizieren und die richtigen Modellparameter auszuwählen, um die Modellleistung zu maximieren. Dies führt nicht nur zu effizienteren Modellen (kürzere Trainingsdauer), sondern verringert auch das Risiko der Überanpassung (Overfitting), da das Modell weniger auf Rauschen in Trainingsdaten basiert [13].

Der Begriff Korrelation oder Zusammenhang kommt aus der Statistik und ist ein Maß für die Stärke und Richtung der linearen Beziehung zwischen zwei Variablen (z.B. Merkmale) [14]. Im Kontext der Modellleistung und der Zeitreihenanalyse kommen verschiedene Korrelationsmethoden zum Einsatz, die in den nachfolgenden Unterkapiteln detailliert erklärt werden.

2.2.1 Spearman-Rangkorrelation

Das Hauptziel der Korrelationsanalyse in dieser Arbeit ist es, die Beziehung zwischen verschiedenen Produktionsparametern und dem Stromverbrauch zu untersuchen. Es hat sich herausgestellt, dass die Beziehung zwischen diesen Variablen nicht linear ist, was bedeutet, dass mit steigender oder fallender Produktionsmenge der Stromverbrauch der Formanlage nicht unbedingt proportional zu- oder abnimmt. Daher wird der Spearman-Rangkorrelationskoeffizient verwendet, der besser geeignet ist, um nichtlineare Beziehungen zu erfassen. Im Gegensatz zum Pearson-Korrelationskoeffizienten, der lineare Beziehungen misst und in diesem Fall weniger geeignet wäre, kann der Spearman-Koeffizient komplexe Beziehungsmuster zwischen den Produktionsparametern und dem Stromverbrauch aufzeigen.

Der Spearman-Rangkorrelationskoeffizient (r_s) bewertet die konsistente Beziehung zwischen zwei Variablen. Weil er nicht die Ausgangsdaten, sondern die Rangplätze der Daten verwendet, ist er unempfindlich gegenüber Ausreißern und nicht

normalverteilten Daten. Ein hoher absoluter Wert von r_s (nahe +1 oder -1) deutet auf eine starke monotone Beziehung hin, die als potenzieller Prädiktor für das Modell dienen kann, selbst wenn die Beziehung nicht linear ist [15]. Die Formel ist gegeben durch:

$$r_s = 1 - \frac{6 \cdot \sum d_i^2}{n \cdot (n^2 - 1)} \in [-1; 1] \tag{2.6}$$

,mit

$d_i =$ Die Differenz zwischen den Rängen der i-ten Beobachtungen

$n =$ Die Anzahl der Beobachtungspaare

2.2.2 Autokorrelation

Die Autokorrelationsfunktion (ACF) ist ein Maß, das die Korrelation eines Signals mit einer verzögerten Version von sich selbst über verschiedene Zeitintervalle (Verzögerungen) hinweg misst. Die ACF gibt einen Gesamteindruck der Korrelation über mehrere Verzögerungsstufen und beschreibt, wie stark die gegenwärtigen Werte einer Zeitreihe durch ihre historischen Werte beeinflusst sind. Die ACF hilft dabei zu identifizieren, ob eine lineare Abhängigkeit über Verzögerungen besteht, was die Beurteilung der Gesamtpersistenz der Korrelation über die Zeit ermöglicht [3]. Die Formel ist wie folgt gegeben:

$$ACF(k) = \frac{E[(z_t - \mu)(z_{t+k} - \mu)]}{\sqrt{E[(z_t - \mu)^2]E[(z_{t+k} - \mu)^2]}} = \frac{E[(z_t - \mu)(z_{t+k} - \mu)]}{\sigma_z^2} \tag{2.7}$$

,mit

$z_t =$ Der Beobachtungswert zum Zeitpunkt t

$\mu =$ Der Mittelwert der gesamten Zeitreihe

$E =$ Der Erwartungswert einer Zufallsvariablen

$k =$ Die Verzögerung (Lag)

$\sigma_z^2 =$ Die Varianz der Zeitreihe

Mit der ACF kann man bestimmen, welcher Parameter im MA(q)-Modell verwendet werden sollte. In einem MA-Modell schneidet die ACF theoretisch nach dem q-ten Lag auf null ab, da die Autokorrelationen von Moving Average-Prozessen nach q Lags keine signifikanten Werte mehr aufweisen sollten. Somit gibt das abrupte Abfallen der ACF auf oder nahe Null nach einer bestimmten Anzahl von Lags einen direkten Hinweis auf die Ordnung q des MA-Modells. Falls das Absinken der ACF auf null sehr langsam erfolgt, deutet dies darauf hin, dass die Zeitreihendaten nicht mit dem MA-Modell kompatibel sind. Entweder eignet sich die Zeitreihe nur für ein AR-Modell, oder die Stationarität der Zeitreihe muss erneut überprüft werden. Diese Information ist entscheidend, um die genaue Anzahl der Lags zu bestimmen, die im MA-Modell berücksichtigt werden müssen, um die Zeitreihe effektiv zu modellieren [16].

2.2.3 Partielle Autokorrelation

Die Partielle Autokorrelationsfunktion (PACF) misst die Korrelation zwischen den Zeitreihenwerten von zwei Zeitpunkten, unter Ausschluss der Einflüsse der Werte zu den dazwischenliegenden Zeitpunkten. Die PACF isoliert den direkten Effekt einer bestimmten zeitlichen Verzögerung und hilft zu bestimmen, welche spezifischen Verzögerungen einen tatsächlichen, direkten Einfluss auf die Zeitreihe haben, im Gegensatz zu einem indirekten Einfluss, der durch die Nähe zu anderen Datenpunkten entsteht [3]. Die Formel ist wie folgt gegeben:

$$PACF(k) = corr(z_t - \hat{z}_t, z_{t-k} - \hat{z}_{t-k}) \tag{2.8}$$

,mit

z_t = Der Beobachtungswert zum Zeitpunkt t

$\hat{z}_t$ = Der vorhergesagte Wert zum Zeitpunkt t

$corr$ = Die Berechnung des Korrelationskoeffizienten

k = Die Verzögerung (Lag)

Die PACF ist nützlich, um zu bestimmen, welche Verzögerungen im AR(p)-Modell verwendet werden sollten, da sie zeigt, welche Verzögerungen tatsächlich signifikanten Einfluss auf die Zeitreihe haben. Für ein AR(p)-Modell zeigt die PACF typischerweise signifikante Korrelationen für die ersten p Lags und schneidet danach auf oder nahe Null ab. Dieser Abfall auf null deutet darauf hin, dass keine weiteren Lags über p hinaus einen signifikanten direkten Einfluss auf die Modellvorhersagen haben. Wenn die PACF nur langsam gegen null konvergiert, lässt das darauf schließen, dass die Zeitreihendaten für ein AR-Modell ungeeignet sind. In diesem Fall könnte die Zeitreihe besser durch ein MA-Modell repräsentiert werden, oder es könnte notwendig sein, die Stationarität der Daten erneut zu evaluieren. Die PACF-Analyse ist entscheidend, um die Anzahl der Lags zu identifizieren, die in das AR-Modell aufgenommen werden sollten, um die Dynamik der Zeitreihe effektiv zu erfassen und zu modellieren [16].

2.3 Hypothesentest

Der Hypothesentest ist ein statistisches Testverfahren, mit dem die Gültigkeit einer Annahme (genannt Nullhypothese) über eine Population basierend auf Stichprobendaten beurteilt wird [17]. In dem Kontext der Korrelationsanalyse wird der p-Wert genutzt, um die Signifikanz der berechneten Korrelationskoeffizienten zu testen. Folgende sind die typischen Hypothesen:

- **Nullhypothese (H0):** Es gibt keinen Zusammenhang zwischen den beiden Variablen X und Y. Das bedeutet, der Korrelationskoeffizient r ist gleich Null.

- **Alternativhypothese (H1):** Es besteht ein Zusammenhang zwischen den beiden Variablen X und Y. Das bedeutet, der Korrelationskoeffizient r ist nicht gleich Null.

Der p-Wert wird verwendet, um die Stärke der Evidenz gegen die Nullhypothese zu messen. Der p-Wert gibt an, wie wahrscheinlich es ist, die beobachteten extremen Ergebnisse zu erzielen, unter der Annahme, dass kein tatsächlicher Zusammenhang zwischen zwei Variablen vorhanden ist (d.h. die Nullhypothese zutrifft) [18]. Ein niedriger p-Wert (< 0,05 als konventionelle Signifikanzschwelle) deutet darauf hin, dass die beobachteten Daten stark genug sind, um die Nullhypothese zugunsten der Alternativhypothese zu verwerfen (siehe Abb. 2.4).

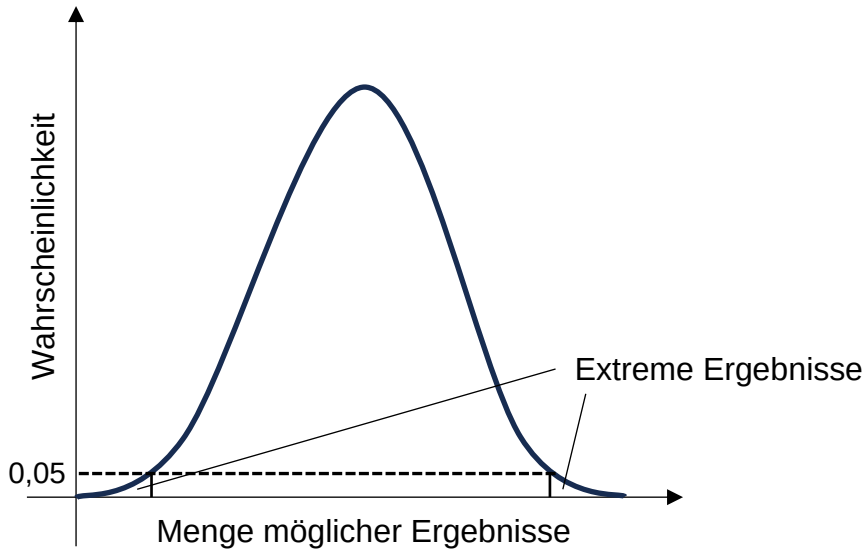

Abb. 2.4: Wahrscheinlichkeitsverteilung mit 0,05 Signifikanzschwelle

Zusätzlich zum traditionellen Hypothesentest wird in dieser Arbeit auch der Augmented Dickey-Fuller-Test (ADF-Test) genutzt, um die Stabilität der Daten zu überprüfen. Der ADF-Test ist ein statistisches Verfahren, mit dem überprüft wird, ob eine Zeitreihe konstante Eigenschaften wie Mittelwert und Varianz über die Zeit aufweist, was eine Grundvoraussetzung für ein ARIMA-Modell ist [19]. Die Formel lautet wie folgt:

$$\Delta y_t = \rho y_{t-1} + \sum_{i=1}^{p} \theta_i \Delta y_{t-i} + e_t \tag{2.9}$$

,mit

y_t = Der aktuelle Wert der Zeitreihe

Δ = Der Differenzoperator ($\Delta y_t = y_t - y_{t-1}$)

ρ = Der Koeffizient von y_{t-1}

θ_i = Die Koeffizienten der verzögerten Differenzen der Zeitreihe

e_t = Der Fehlerterm

Mit dem ADF-Test untersucht man die Zeitreihendaten unter der Annahme, dass sie nicht stationär sind (d.h. die Nullhypothese zutrifft und ρ = 1 ist). Ein p-Wert unter 0,05 deutet darauf hin, dass ρ signifikant kleiner als 1 ist und dies bedeutet, dass die Zeitreihe als stationär betrachtet wird. Dies führt dazu, dass die Nullhypothese verworfen und die Alternativhypothese, dass die Daten stationär sind, akzeptiert wird. Daher ist kein Differenzierungsgrad (I-Modell) erforderlich. Dies zeigt, dass die Daten die Anforderungen für eine Modellierung ohne Differenzierungsanteil, also für ein ARMA-Modell, erfüllen [19].

2.4 Modell-Evaluierung

Um die Modellleistung so unparteiisch wie möglich bewerten zu können, werden die drei Vorhersagemodelle mithilfe einer Cross-Validation evaluiert, die mehrmalige Bewertungen der Modelle ermöglicht. Dabei wird die Modellleistung mit drei verschiedenen Evaluierungsmetriken quantifiziert, sodass ein direkter und anschaulicher Vergleich zwischen den verschiedenen Modellen möglich ist. Die in dieser Arbeit verwendeten Methoden der Cross-Validation und die Evaluierungsmetriken werden im nächsten Unterkapitel vorgestellt.

2.4.1 Cross-Validation

Cross-Validation ist eine statistische Methode, die Datenstichproben in kleinere Teilmengen aufteilt und verifiziert. Bei einer Cross-Validation wird auch die Modellleistung bzw. Vorhersagegenauigkeit präziser dargestellt [20]. Eine einmalige Bewertung verifiziert die Modellleistung nur einmalig mit einem kleinen Teil der Trainingsdaten als Validierungsdaten (siehe Abb. 2.5). Im Gegensatz dazu ermöglicht Cross-Validation eine wiederholte Bewertung und Validierung sowie kontinuierliches Training.

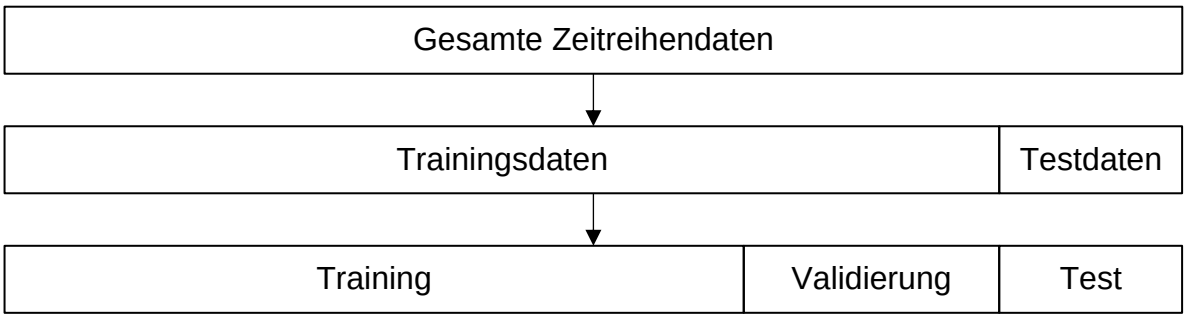

Abb. 2.5: Einmalige Modellbewertung

Für unterschiedliche Anwendungen kommen verschiedene Typen der Cross-Validation zum Einsatz. Im speziellen Kontext der Vorhersagemodelle für Zeitreihendaten wird in dieser Arbeit die Variante TimeSeriesSplit verwendet. Diese Methode beginnt mit einer kleinen Teilmenge von Daten, für die anschließend

Prognosen erstellt und überprüft werden. Die prognostizierten Datenpunkte werden dann in den nächsten Trainingsdatensatz aufgenommen, um weitere Prognosen zu generieren (siehe Abb. 2.6).

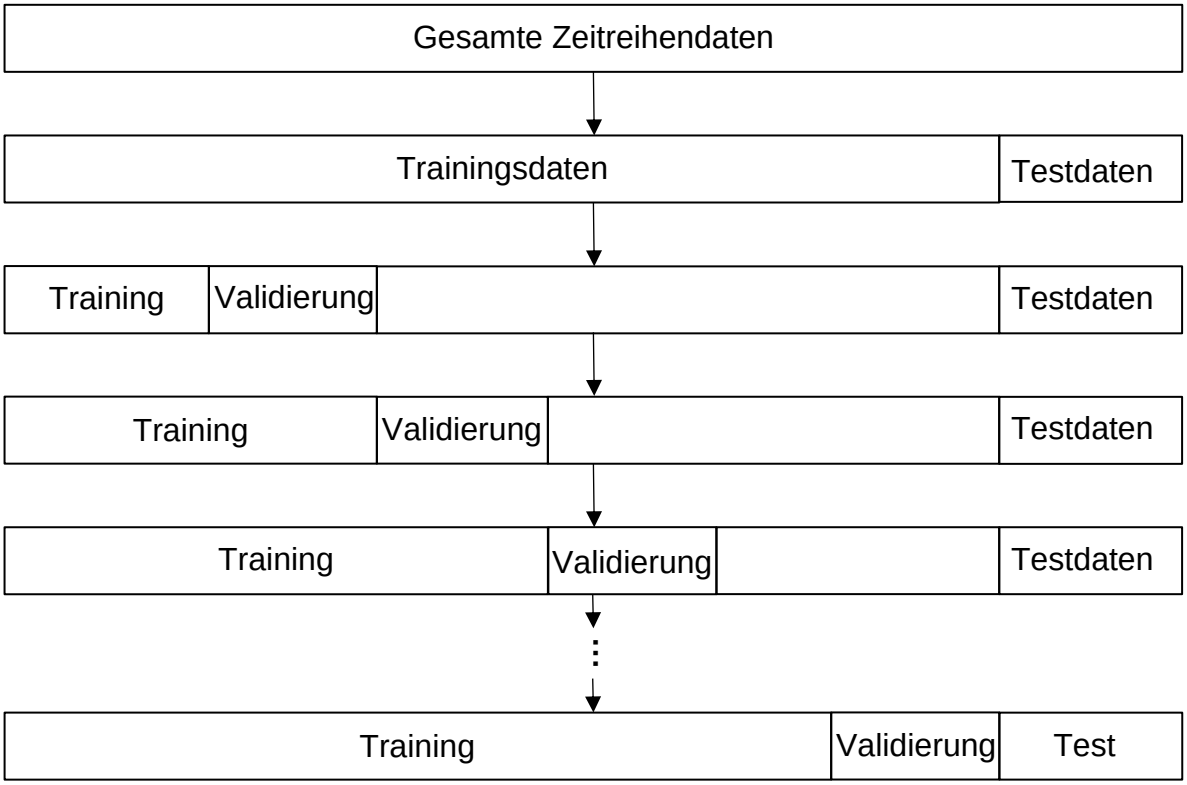

Abb. 2.6: TimeSeriesSplit-Methode der Cross-Validation

Die TimeSeriesSplit-Methode ist ideal für Vorhersagemodelle in Zeitreihendaten aufgrund der bestehenden zeitlichen Verbindung (Abhängigkeit) zwischen Vergangenheit und Zukunft. Dies impliziert, dass das Modell nicht mit zukünftigen Daten trainiert werden sollte, um Vorhersagen für vergangene Zeiträume zu generieren. Stattdessen sollen die Trainingsdaten nur aus Beobachtungen stammen, die zeitlich vor den Validierungsdaten liegen [21] [22].

In der weiteren Evaluierung der Modellleistung werden die Daten zunächst in Trainings- und Testdaten aufgeteilt. Anschließend werden die Trainingsdaten mithilfe der TimeSeriesSplit-Methode genutzt, um das Modell mehrmals zu trainieren und zu validieren. Dies soll eine Mittelwertbewertung aus verschiedenen Validierungen generieren und das Overfitting vermeiden, indem das Modell über mehrere Zeiträume trainiert wird, anstatt nur in einem einzelnen Zeitraum. Zum Schluss dienen die übrigen Testdaten zur Beurteilung der Vorhersagegenauigkeit, da sie im Trainingsprozess des Modells nicht verwendet wurden und somit für das Modell eine ungewisse Zukunft darstellen.

2.4.2 Evaluierungsmetriken

Bei der Modellierung ist die genaue Evaluierung der Modellleistung wichtig, um die geeignetsten Modelle für die Stromverbrauchsmodellierung auswählen zu können [23]. In den nächsten Absätzen werden die drei Metriken, die für diese Arbeit angewendet werden, zur Beurteilung der Modellgenauigkeit vorgestellt:

- Mean Absolute Error (MAE)
- Root Mean Square Error (RMSE)
- R-Squared (R2)

MAE

Der MAE wird definiert als der Durchschnitt der absoluten Differenzen zwischen den vorhergesagten und den tatsächlichen Werten [24]. Mathematisch wird dieser wie folgt formuliert:

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |x_i - \hat{x}_i| \tag{2.10}$$

,mit

x_i = Die tatsächlichen Werte der Zielvariablen
$\hat{x}_i$ = Die vorhersagten Werte der Zielvariablen
n = Die Anzahl der Messungen

RMSE

Der RMSE ist eine weitere häufig verwendete Metrik, die die durchschnittliche Größe der Fehler misst, indem die quadrierten Abweichungen zwischen der Vorhersage und dem tatsächlichen Wert summiert und dann die Wurzel daraus gezogen wird [24]. Im Gegensatz zum MAE gewichtet der RMSE größere Fehler stärker, was den RMSE empfindlicher gegenüber Ausreißern macht. Der RMSE ist definiert als:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (x_i - \hat{x}_i)^2} \tag{2.11}$$

,mit

x_i = Die tatsächlichen Werte der Zielvariablen
$\hat{x}_i$ = Die vorhersagten Werte der Zielvariablen
n = Die Anzahl der Messungen

R2

R2 ist eine statistische Metrik, die angibt, welcher Anteil der Varianz in der abhängigen Variable durch das Modell erklärt wird [24]. Ein R2-Wert von 1 bedeutet, dass das Modell die Daten perfekt erklärt, während ein Wert von 0 darauf hindeutet, dass das Modell nicht besser ist als ein einfaches Modell, das nur den Mittelwert der Daten verwendet. R2 ist definiert als:

$$R2 = 1 - \frac{\sum_{i=1}^{n}(x_i - \hat{x}_i)^2}{\sum_{i=1}^{n}(x_i - \overline{x})^2} \tag{2.12}$$

,mit

$\overline{x}$ = Der Mittelwert der tatsächlichen Werte

x_i = Die tatsächlichen Werte der Zielvariablen

$\hat{x}_i$ = Die vorhersagten Werte der Zielvariablen

n = Die Anzahl der Messungen

2.5 Python-Pakete

Die Programmiersprache Python ist eine interpretierte Hochsprache, die besonders in der wissenschaftlichen Datenanalyse und im Bereich der künstlichen Intelligenz (KI) weltweit angewendet wird. Bei der Entwicklung von ML-Modellen ermöglicht Python den Einsatz vorprogrammierter Bibliotheken, sodass Entwickler Algorithmen nicht von Grund auf selbst implementieren müssen. Dank dieser umfangreichen Bibliotheken können die Entwickler sich mehr auf die Analyse der Daten und die Optimierung der Modellparameter konzentrieren [25]. Die in dieser Arbeit verwendeten Python-Pakete werden in den nächsten Absätzen beschrieben.

NumPy

NumPy bietet Arrays mit mehreren Achsen, die von einer umfangreichen Sammlung von Funktionen unterstützt werden, um verschiedene mathematische Operationen auf diesen Arrays auszuführen. Arrays in NumPy sind strukturierte Datensätze, die entlang mehrerer Dimensionen angeordnet sind und die Basis für die Implementierung komplexer mathematischer Strukturen bilden. Diese Arrays spielen in dieser Arbeit eine wichtige Rolle für die Datenspeicherung und sind unerlässlich für effiziente Matrixoperationen im Bereich des DL [26].

Pandas

Pandas ermöglicht das Verwalten komplexer Datentabellen verschiedener Typen und Zeitreihen, was NumPy-Arrays nicht verarbeiten können. Mit den spezifischen Datenstrukturen, insbesondere DataFrames, die in dieser Arbeit verwendet werden,

kann man Daten mühelos und problemlos aus verschiedenen Quellen importieren. Daraufhin können die Daten frei zugeschnitten, fehlende Elemente behandelt, hinzugefügt, umgeformt, umbenannt und ähnlich wie in Excel-Tabellen visualisiert werden [26].

Matplotlib

Matplotlib ist eine Python-Bibliothek für MATLAB-ähnliche 2D-Diagramme, die alle Komponenten enthält, um qualitativ hochwertige Grafiken aus Daten zu erstellen und diese interaktiv zu visualisieren. Darüber hinaus ermöglicht sie ausführliche Anpassungsoptionen durch Plotarten, Achsenformate, Schrifteigenschaften und mehr, mit denen man die Visualisierung präziser gestalten kann [26].

Scikit-learn

Scikit-learn bildet das Kernstück für ML und Datenanalysen in Python. Diese Bibliothek bietet eine breite Palette an Werkzeugen und Funktionen, die nicht nur Datenvorverarbeitung und Cross-Validation abdecken, sondern auch die Möglichkeiten zur Fehlermessung und zur Evaluierung der Modellleistung (MAE, RMSE und R2) bieten [25].

Statsmodels

Statsmodels ist ein Python-Paket für statistische Berechnungen und umfasst unter anderem Funktionen wie Hypothesentests und Zeitreihenanalysen und ermöglicht komplexe Analysen durch die Integration mit anderen Python-Bibliotheken wie NumPy und Pandas. Darüber hinaus bietet Statsmodels die Unterstützung für das ARIMA-Modell zur Modellierung und Prognose der Zeitreihen [27].

SciPy

SciPy ist ein Python-Paket, das speziell für wissenschaftliche und technische Berechnungen entwickelt wurde. Es bietet eine breite Palette an mathematischen Algorithmen und Funktionen. Für diese Bachelorarbeit ist insbesondere die Funktion zur Berechnung der Korrelation von Bedeutung [28].

PyTorch

PyTorch ist ein Open-Source-Framework für den Forschungsbereich DL, welcher eine flexible Programmierschnittstelle für die Anpassung und Implementierung der neuronalen Netzwerkarchitekturen wie LSTM bietet. In PyTorch müssen die Daten in Tensoren (ähnlich wie NumPy-Arrays) umgewandelt werden, damit die Informationen als numerische Dateneinheiten für die Berechnung genutzt werden können [29].

XGBoost

XGBoost ist eine optimierte Gradient-Boosting-Bibliothek in Python, mit der man den ML-Algorithmus XGBoost direkt anwenden kann, ohne die Algorithmen, die dahinter stecken, selbst implementieren zu müssen [30].

3 Vorverarbeitung der Zeitreihendaten

Das nachfolgende Kapitel erläutert die Vorbereitung der Daten, beginnend damit, wie die Rohdaten der Firma Fondium in die Analyseumgebung importiert werden. Es wird beschrieben, wie die importierten Daten in zeitbezogene Daten mit indexbezogenen Datumsangaben transformiert und bereinigt werden können. Abschließend wird untersucht, welche Parameter den Zielwert (Stromverbrauch) signifikant beeinflussen und sich daher als Eingabe für das Modell eignen.

3.1 Laden der Daten

Die verfügbaren Rohdaten der Formanlage der Firma Fondium sind in einer Comma-Separated-Values (CSV)-Datei gespeichert, die nicht direkt in der Analyseumgebung bearbeitet werden kann. Um diese CSV-Datei in die Entwicklungsumgebung (z.B. PyCharm) zu laden und weitere Bearbeitungen sowie Analysen der Daten durchführen zu können, kann die Python-Bibliothek Pandas verwendet werden. Mithilfe der Funktion *read_csv* wird die CSV-Datei als ein DataFrame in die Python-Umgebung importiert (siehe Abb. 3.1).

```
import pandas as pd

df = pd.read_csv(FILE_PRODUKTIONSDATEN_2023)
print(df.head())
print(df.tail())
print(df.info())
```

Abb. 3.1: Datenimport mit Pandas

DataFrame ist ein spezielles Datenformat in Pandas, welches aus zweidimensionalen Tabellen mit einer Reihe benannter und geordneter Spalten besteht. Mithilfe der Methoden *head()* und *tail()* können die ersten fünf bzw. die letzten fünf Zeilen der geladenen Daten in der Ausgabe der Entwicklungsumgebung angezeigt werden (siehe Abb. 3.2). Zudem lässt sich erkennen, dass die ursprünglichen Daten vom Unternehmen in 5-Minuten-Intervallen dokumentiert wurden. Diese Produktionsdaten wurden über einen Zeitraum von fast 11 Monaten, vom 1. Januar 2023 bis zum 21. November 2023, erfasst.

	Zeitstempel	Stromverbrauch	...	Produktion	Störung
0	2023-01-01 00:00:00	47.596	...	0	1
1	2023-01-01 00:05:00	50.325	...	0	1
2	2023-01-01 00:10:00	48.062	...	0	1
3	2023-01-01 00:15:00	49.374	...	0	1
4	2023-01-01 00:20:00	47.727	...	0	1

	Zeitstempel	Stromverbrauch	...	Produktion	Störung
93308	2023-11-20 23:40:00	264.28	...	1	0
93309	2023-11-20 23:45:00	272.49	...	1	0
93310	2023-11-20 23:50:00	255.12	...	1	0
93311	2023-11-20 23:55:00	201.36	...	0	1
93312	2023-11-21 00:00:00	262.57	...	1	0

Abb. 3.2: Vorschau der Daten

Mit der Methode *info()* lässt sich ein Überblick über die gesamten Daten gewinnen. Die Daten umfassen insgesamt 7 Variablen (Columns), darunter den Zeitstempel, den Stromverbrauch und weitere Produktionsparameter, beispielsweise die Kastenanzahl, das Gießgewicht und das durchschnittliche Gießgewicht pro Kasten. Insgesamt enthalten die Daten 93313 Zeilen. Der Überblick enthält auch andere detaillierte Informationen, etwa welche Datentypen in Pandas für die ursprünglichen Variablen aus der CSV-Datei initialisiert wurden und wie viel Speicherplatz das DataFrame beansprucht (siehe Abb. 3.3).

```
RangeIndex: 93313 entries, 0 to 93312
Data columns (total 7 columns):
 #   Column                  Non-Null Count  Dtype
---  ----------              --------------- ---------
 0   Zeitstempel             93313 non-null  object
 1   Stromverbrauch          93313 non-null  float64
 2   Kasten_Anzahl           93313 non-null  int64
 3   Gießgewicht             93313 non-null  float64
 4   durchschn_Gießgewicht   93313 non-null  float64
 5   Produktion              93313 non-null  int64
 6   Störung                 93313 non-null  int64
dtypes: float64(3), int64(3), object(1)
memory usage: 5.0+ MB
```

Abb. 3.3: Übersicht der DataFrame-Struktur

3.2 Verwendung der Datums- und Zeitindizes

Obwohl die Daten jetzt erfolgreich als DataFrame in die Entwicklungsumgebung geladen werden können, müssen sie weiter standardisiert werden, um sie in Zeitreihendaten zu transformieren. Das bedeutet, dass die Indizes des DataFrames in ein spezifisches Zeitstempelformat umgewandelt werden müssen, damit dieser DataFrame für spätere Zeitreihenanalysen zur Verfügung steht.

Mit der Pandas-Bibliothek können zunächst die Zeitvariablen (Zeitstempel) mithilfe der Funktion *to_datetime* in ein Pandas-spezifisches Zeitformat konvertiert werden. Anschließend können diese formatierten Zeitstempel mit dem automatisch von Pandas erzeugten numerischen Index ersetzt werden. Schlussendlich erhält man einen neuen DataFrame, der als Zeitreihendaten formatiert ist (siehe Abb. 3.4).

```
df['Zeitstempel'] = pd.to_datetime(df['Zeitstempel'])
df = df.set_index('Zeitstempel')
```

Abb. 3.4: Zeitstempel-Konvertierungsprozess

Nachdem der Zeitstempel als Index festgelegt wurde, können mit den Methoden *head()* und *tail()* die ersten fünf und letzten fünf Zeilen des DataFrames aufgerufen werden. Dies dient dazu, zu überprüfen, ob der alte numerische Index erfolgreich in das Pandas-spezifische Zeitformat umgewandelt wurde (siehe Abb. 3.5).

	Stromverbrauch	Kasten_Anzahl	...	Produktion	Störung
Zeitstempel					
2023-01-01 00:00:00	47.596	0	...	0	1
2023-01-01 00:05:00	50.325	0	...	0	1
2023-01-01 00:10:00	48.062	0	...	0	1
2023-01-01 00:15:00	49.374	0	...	0	1
2023-01-01 00:20:00	47.727	0	...	0	1

	Stromverbrauch	Kasten_Anzahl	...	Produktion	Störung
Zeitstempel					
2023-11-20 23:40:00	264.28	12	...	1	0
2023-11-20 23:45:00	272.49	14	...	1	0
2023-11-20 23:50:00	255.12	14	...	1	0
2023-11-20 23:55:00	201.36	0	...	0	1
2023-11-21 00:00:00	262.57	14	...	1	0

Abb. 3.5: Überprüfung der Zeitstempel-Indizierung

3.3 Datenbereinigung

Die in der Praxis gesammelten Daten sind oft unstrukturiert und fehlerhaft, was zu Problemen führen kann, da Modelle gegenüber solchen Daten, wie falsch eingetragenen extremen Werten oder fehlenden Werten, empfindlich reagieren. Daher können die standardisierten Zeitreihendaten noch nicht für die Modellierung verwendet werden, solange Ausreißer den Lernprozess des Modells verzerren könnten.

Um fehlerhaft eingetragene, extrem hohe Stromverbrauchswerte zu identifizieren, wird ein Histogramm des Stromverbrauchs zur Analyse der Ausreißer erstellt. Dabei ist zu erkennen, dass oberhalb eines Stromverbrauchs von 325 Kilowattstunden (kWh) keine signifikanten Häufigkeiten mehr vorkommen, was darauf hindeutet, dass der mögliche maximale Stromverbrauch der Anlage bei 325 liegt. Daher werden die Daten, deren Stromverbrauch höher als 325 ist, als Ausreißer behandelt und entfernt (siehe Abb. 3.6).

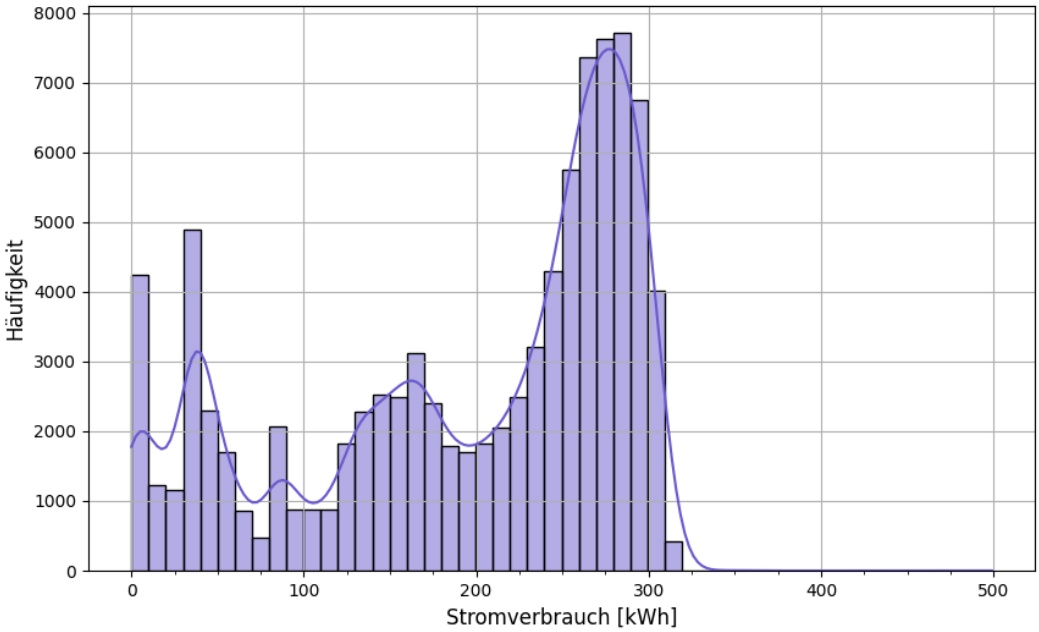

Abb. 3.6: Histogramm der Stromverbrauchsverteilung

Zur Vereinfachung der Datenbereinigung und aufgrund zahlreicher fehlender Werte werden in dieser Arbeit die von Pandas als NaN (Not a Number) markierten fehlenden Werte ausgeschlossen. Es wurde festgestellt, dass der Stromverbrauch aus der Produktion niemals Null sein kann, selbst wenn die Formanlage während eines Produktionsausfalls (wie beispielsweise vom 05. bis 20. August 2023) komplett abgeschaltet ist. Daher werden fehlerhaft als Null eingetragene Werte für den Stromverbrauch ebenfalls entfernt (siehe Abb. 3.7).

```
df.dropna(inplace=True)
df = df[df['Stromverbrauch'] != 0]
df = df[df['Stromverbrauch'] <= 325]
```

Abb. 3.7: Entfernen von Ausreißern und Nullwerten im Stromverbrauch

3.4 Feature Engineering

Feature Engineering ist ein Prozess in der ML-Modellbildung, bei dem Wissen aus speziellen Bereichen genutzt wird, um wichtige Merkmale aus Rohdaten zu extrahieren. Dies trägt zur Effizienzsteigerung des Trainingsprozesses bei und erhöht die Qualität der Ergebnisse im Vergleich dazu, alle Rohdaten ohne gezielte Feature-Auswahl zu verwenden [31].

Für eine effektive Modellierung sollten die Eingabeparameter (Features) sorgfältig ausgewählt werden, sodass das Modell nicht durch alle Produktionsparameter überlastet wird, was die Mustererkennung erschweren könnte. Für eine umsetzbare künftige Produktionsplanung werden die Kastenanzahl, das Gießgewicht und das durchschnittliche Gießgewicht pro Kasten als mögliche Produktionsparameter berücksichtigt. Daher wird zunächst die Korrelation dieser drei Parameter mit der Zielvariable (Stromverbrauch) untersucht, um den aussagekräftigsten Produktionsparameter für die effiziente Modellierung identifizieren zu können.

Die anderen zwei Parameter, Produktion und Störung, sind für diese Arbeit nicht geeignet, da ein Produktionsplaner nicht vorhersagen kann, wann die Formanlage unerwartet stillsteht oder eine Störung auftritt. Im Gegensatz dazu lässt sich planen, wie viele Produkte (Kastenanzahl) im nächsten Monat für Kunden hergestellt werden müssen. Die Spearman-Korrelation (siehe Formel 2.6) der drei planbaren Parameter mit dem Stromverbrauch wird mithilfe einer Korrelationsmatrix und des zugehörigen p-Wertes visualisiert, um den aussagekräftigsten Produktionsparameter ermitteln zu können (siehe Abb. 3.8).

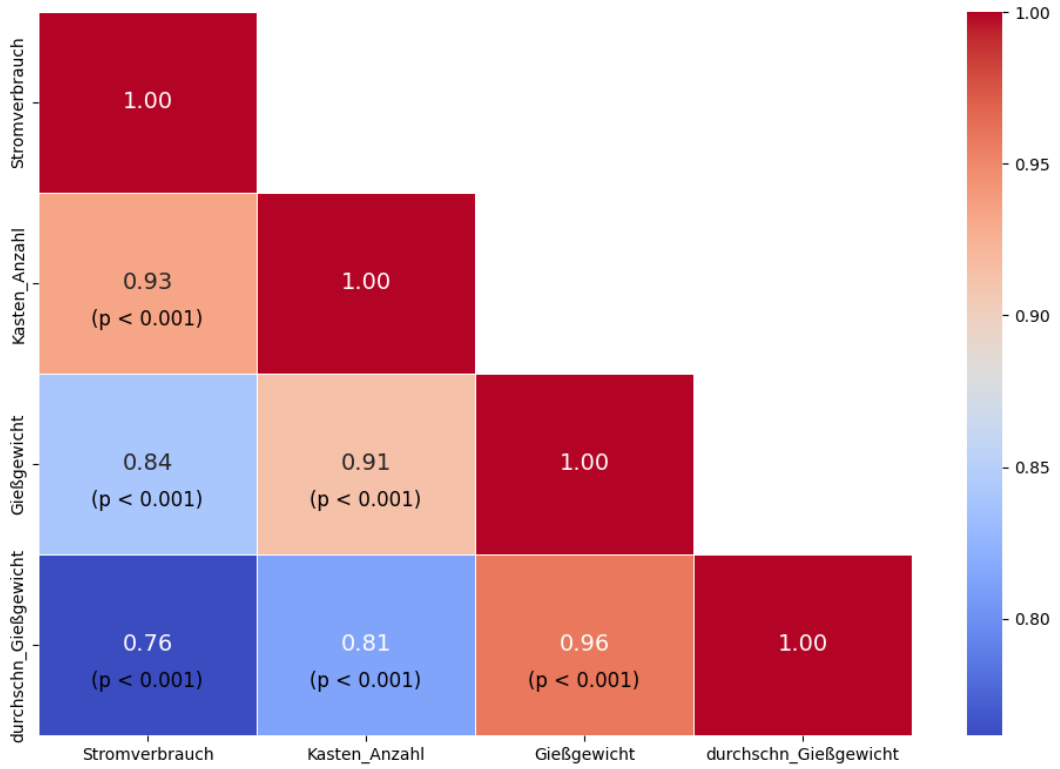

Abb. 3.8: Spearman-Korrelationsmatrix der Produktionsdaten mit p-Werten

Die obige Korrelationsmatrix zeigt, dass die Kastenanzahl der aussagekräftigste Parameter ist, welcher den Stromverbrauch signifikanter beeinflusst als das Gießgewicht und das durchschnittliche Gießgewicht pro Kasten. Die sehr niedrigen p-Werte (p < 0,001) bestätigen, dass die Korrelationsergebnisse statistisch signifikant und nicht zufällig sind, d.h., dass dieser Zusammenhang tatsächlich besteht. Dies ist insbesondere auf die hinreichend große Datenmenge zurückzuführen. Aufgrund der Vereinfachung der Produktionsplanung und der Reduzierung der Modellkomplexität wird die Kastenanzahl in dieser Arbeit als einziger Produktionsparameter für die Modellierung verwendet.

Neben den Produktionsparametern können auch zeitliche Elemente als exogene Variablen in der Modellierung verwendet werden, wie zum Beispiel der Arbeitstag, Feiertage (einschließlich Sonntage) in Baden-Württemberg oder Produktionsausfälle. Dabei wird der in 5-Minuten-Intervallen erfasste Stromverbrauch in durchschnittliche tägliche Verbrauchswerte umgewandelt, um zu überprüfen, ob die Stromschwankungen von einem Arbeitstag, einem Feiertag oder von einem Produktionsausfall abhängen (siehe Abb. 3.9).

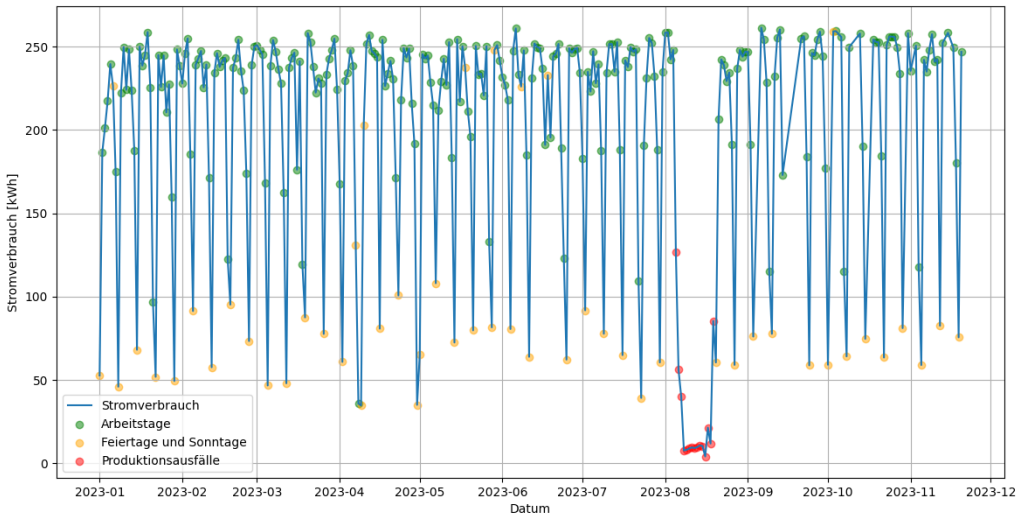

Abb. 3.9: Vergleich des täglichen Stromverbrauchs: Feiertage, Arbeitstage und Produktionsausfälle

In Abb. 3.9 ist ersichtlich, dass an regulären Arbeitstagen der Stromverbrauch im Allgemeinen höher ist als an Feiertagen und Sonntagen. Dennoch gibt es einige Feiertage, an denen der Stromverbrauch auch hoch ist. Das zeigt, dass das Unternehmen trotz mancher Feiertage die Produktionsanlagen in Betrieb hat und kontinuierlich Produkte herstellt. Nach diesen Beobachtungen eignen sich Arbeitstage daher besser als Feiertage um als zeitliche exogene Variablen verwendet zu werden. Es kann auch bemerkt werden, dass der Stromverbrauch an den Tagen vom 05. bis 20. August 2023, an denen die gesamte Produktion stillgelegt ist, deutlich niedriger ist als an Feiertagen. Deswegen können auch Produktionsausfälle als externe zeitliche Variablen genutzt werden, um die Schwankungen im Energieverbrauch genauer zu modellieren. Des Weiteren können die stündlichen Schwankungen während eines Arbeitstages mit dem folgenden Boxplot analysiert werden:

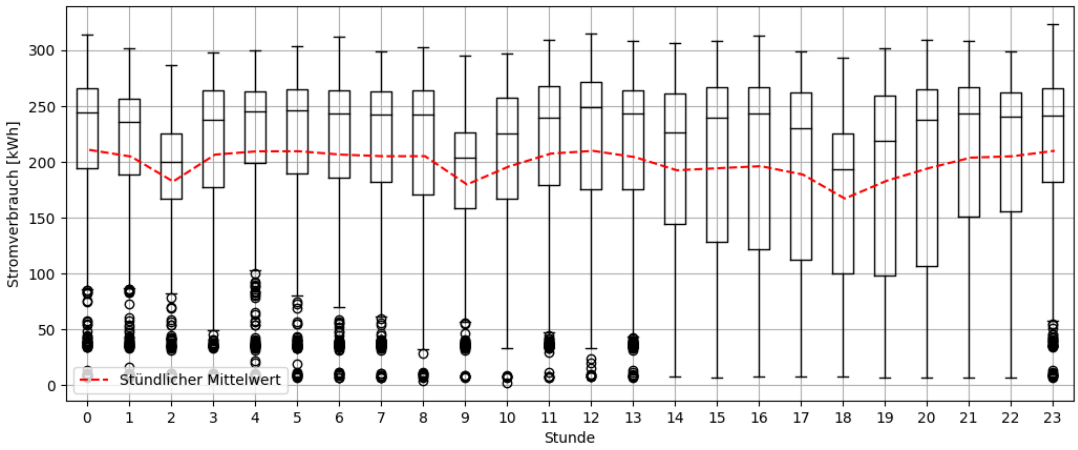

Abb. 3.10: Boxplots des stündlichen Stromverbrauchs

In Abb. 3.10 ist eine leichte Schwankung im durchschnittlichen stündlichen Stromverbrauch erkennbar, besonders auffällig sind die Rückgänge um die Zeiten der Schichtwechsel (täglich um 2, 9 und 18 Uhr). Der Stromverbrauch steigt nach den Schichtwechseln an und sinkt dann vor dem nächsten Schichtwechsel wieder. Diese Beobachtung legt nahe, dass die Stunde als zeitliche Variable für eine Feinjustierung des stündlichen Stromverbrauchs in der Modellierung verwendet werden kann. Darüber hinaus ist es aufschlussreich, diese drei Arbeitsschichten näher zu analysieren, um festzustellen, ob sie einen gleichmäßigen Stromverbrauch während der Nacht, tagsüber und am Abend erzeugen (siehe Abb. 3.11).

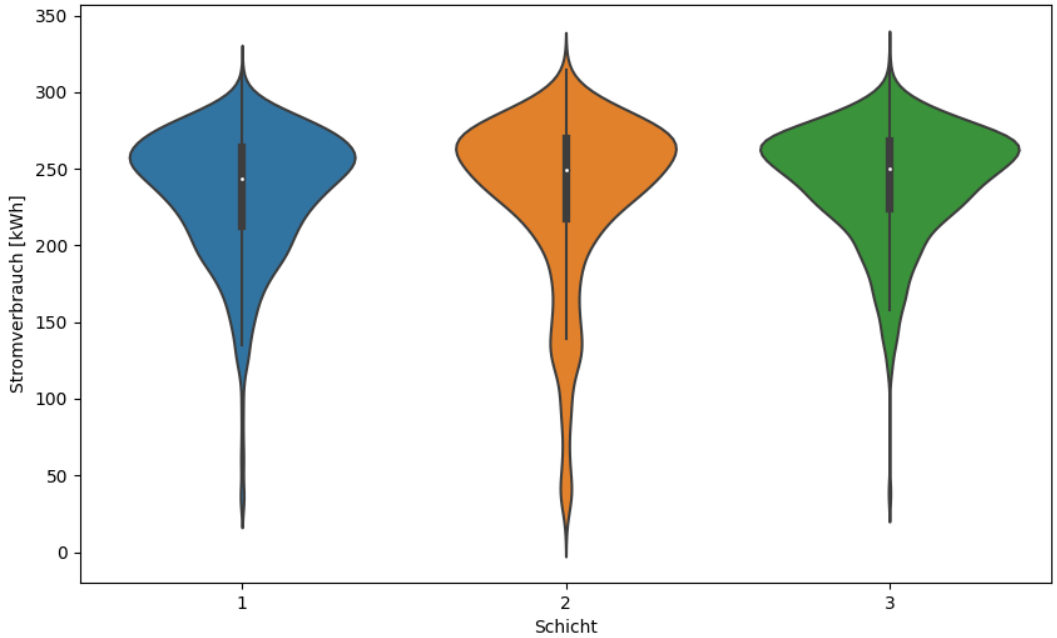

Abb. 3.11: Violinplot des Stromverbrauchs nach Schicht

In Abb. 3.11 ist ersichtlich, dass die drei verschiedenen Schichten sehr ähnliche Mittelwerte und Verteilungen des Stromverbrauchs aufweisen, was auf eine gleichförmige Produktion hinweist. Dies bedeutet, dass sie einen generell konstanten Stromverbrauch erzeugen. Aus diesem Ergebnis lässt sich ableiten, dass die Schicht als Eingabeparameter in der Modellierung ausgeschlossen werden kann, da sie keinen signifikanten Einfluss auf den Stromverbrauch hat.

Zusammenfassend lässt sich sagen, dass für eine effektive Modellierung und eine Vereinfachung der Produktionsplanung in der weiteren Modellentwicklung die vier Eingabeparameter Kastenanzahl, Arbeitstag, Produktionsausfall und Stunde angewendet werden. Diese vier Parameter sind laut Analyse die aussagekräftigsten und planbarsten, mit denen man eine einfach durchführbare Produktionsplanung erstellen und gleichzeitig den künftigen Stromverbrauch effizient modellieren kann. Vor der Entwicklung des Modells können die vier ausgewählten Eingabeparameter und die

Zielvariable Stromverbrauch als globale Variablen im Skript deklariert werden, um die Modellimplementierung zu vereinfachen (siehe Abb. 3.12).

```
FEATURES = ['Kasten_Anzahl',
            'Arbeitstag',
            'Produktionsausfall',
            'Stunde']
TARGET = 'Stromverbrauch'
```

Abb. 3.12: Übersicht der Features und Zielvariablen im Modell

Aufgrund zahlreicher fehlender Werte in den ursprünglichen Zeitreihendaten und den Anforderungen an die Durchführbarkeit der Produktionsplanung sind Daten, die in 5-Minuten-Intervallen erfasst werden, für die Modellierung nicht optimal geeignet. Deshalb werden diese Zeitreihendaten auf eine stündliche Auflösung umgestellt, um dem Produktionsplaner die Erstellung eines praktikablen stündlichen Plans zu ermöglichen. Genauer gesagt ist die Planung der Produktion auf stündlicher Basis realisierbarer und besser planbar als in 5-Minuten-Intervallen. Darüber hinaus führt diese Umstellung zu einer kürzeren Trainingsdauer des Modells und zu einer Reduzierung der gesamten Anzahl fehlender Datenpunkte, weil die Gesamtanzahl der Datenpunkte durch die Reduktion auf ein Zwölftel ihrer ursprünglichen Menge pro Stunde verringert wird.

Für die Umstellung wird die Zielvariable Stromverbrauch, die bisher in 5-Minuten-Intervallen erfasst wurde, in den durchschnittlichen stündlichen Stromverbrauch umgewandelt. Die exogene Variable Kastenanzahl wird als stündliche Summe berechnet. Die Features Arbeitstag, Produktionsausfall und Stunde bleiben unverändert, da sie in gleichen oder größeren Zeiteinheiten als eine Stunde erfasst werden und daher durch die Umstellung nicht beeinflusst sind.

3.5 Datenaufteilung

In der weiteren Modellbildung wird nur ein Teil der vorhandenen Daten als Trainingsdaten genutzt, sodass die Leistung des Modells später mit ungenutzten Daten getestet werden kann. Aus diesem Grund werden die Zeitreihendaten im Verhältnis von 90% Trainingsdaten zu 10% Testdaten geteilt (siehe Abb. 3.13).

```
split_index = int(len(df) * 0.9)
train_data = df.iloc[:split_index]
test_data = df.iloc[split_index + 1:]
```

Abb. 3.13: Aufteilung der Zeitreihendaten in Trainings- und Testdaten

Dadurch kann die Modellleistung nach dem Training mit 90% der Daten anhand der verbleibenden 10% der unbekannten Daten bewertet werden. Mit dieser Aufteilung erhält man ungefähr 10 Monate Stromverbrauchsdaten (vom 1. Januar bis zum 18. Oktober) als Trainingsdaten und einen Monat als Testdaten (vom 18. Oktober bis zum 21. November). Für das Modelltraining wird die in Kapitel 2.4.1 erklärte TimeSeriesSplit-Methode mit 10 Splits als Cross-Validation integriert, wobei jeder Split ungefähr einem Monat entspricht. Dies bedeutet, dass das Modell in Monatseinheiten trainiert und bewertet wird. Die Visualisierung der Aufteilung ist in der folgenden Abbildung dargestellt:

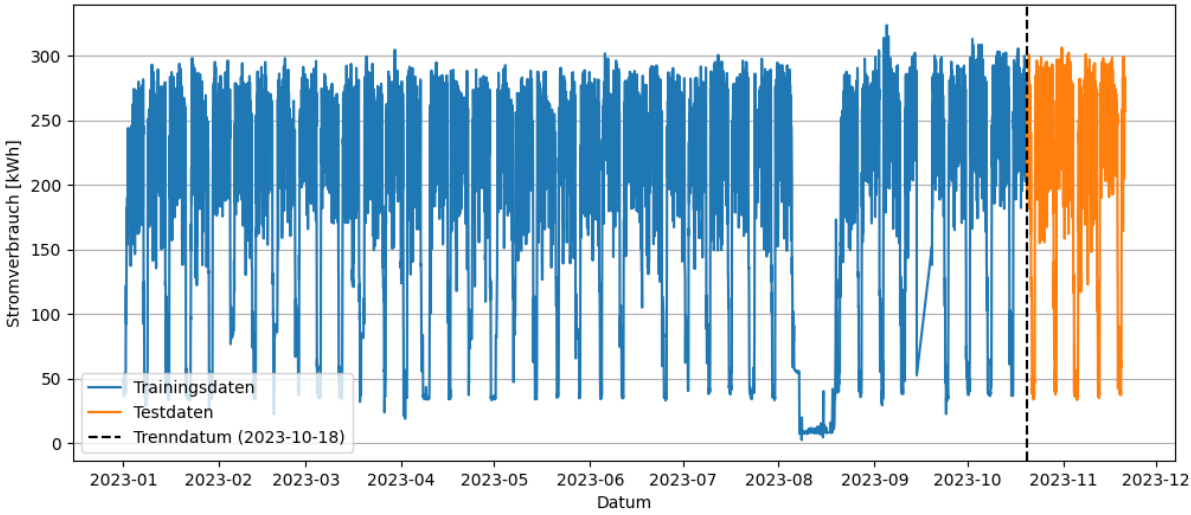

Abb. 3.14: Visualisierung der Stromverbrauchsverteilung

Nachdem die Rohdaten bereinigt und in einen DataFrame umgewandelt wurden, können diese Zeitreihendaten zusammen mit den untersuchten aussagekräftigen Merkmalen für die Modellierung verwendet werden. Dabei werden die Kastenanzahl, der Arbeitstag, der Produktionsausfall und die Stunde als Merkmale verwendet, um die Zielvariable Stromverbrauch zu modellieren. Die detaillierte Entwicklung der Modelle wird im nächsten Kapitel erläutert.

4 Entwicklung der Modelle

Das nachfolgende Kapitel beschreibt den Aufbau des Programmcodes der drei Vorhersagemodelle (ARIMA, LSTM und XGBoost), beginnend mit den notwendigen Schritten zur Modellimplementierung. Dazu gehören zum Beispiel das Training der Modelle für Zeitreihendaten in Python. Es wird auch erklärt, wie die Modellparameter angepasst werden können, um die Modellleistung bei minimaler Trainingsdauer zu maximieren. Abschließend wird die Simulation der Vorhersage gezeigt und visualisiert, um die praktische Anwendung des Modells zu demonstrieren.

4.1 ARIMA

Dank der vorprogrammierten Python-Bibliothek Statsmodels muss für diese Arbeit der Code der gesamten ARIMA-Formel nicht von Grund auf neu geschrieben werden, um das ARIMA-Modell einsetzen zu können. Stattdessen kann man die Python-Bibliothek Statsmodels in die Entwicklungsumgebung importieren, um alle Funktionen zu erhalten, die für die Modellierung des Stromverbrauchs benötigt werden.

4.1.1 Auswahl der Parameter

Bevor das ARIMA-Modell für Vorhersagen verwendet werden kann, muss zunächst die Stationarität der Stromverbrauchsdaten überprüft werden, insbesondere ob diese Zeitreihendaten Trends oder saisonale Elemente enthalten. Dazu kann der in Kapitel 2.3 erwähnte ADF-Test verwendet werden, um die Notwendigkeit des I(d)-Modells zu prüfen [32]. Das Python-Paket Statsmodels bietet die Funktion *adfuller*, mit der man die Stationarität mittels des ADF-Tests überprüfen kann (siehe Abb. 4.1).

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(df[TARGET])
labels = ['ADF Test Statistic', 'p-value']
for value, label in zip(result, labels):
    print(label + ' : ' + format(value))
```

Abb. 4.1: Stationaritätstest der Zeitreihendaten mittels ADF-Test

Nach der Code-Ausführung erhält man den p-Wert des ADF-Tests. In der Ausgabe wird ein p-Wert von etwa $6{,}21 \times 10^{-18}$ angezeigt, der praktisch Null entspricht. Dies deutet darauf hin, dass die ursprüngliche Datenserie weder trend- noch saisonabhängig ist. Der Testwert von ρ = -10,19 bestärkt die Ablehnung der Nullhypothese (siehe Abb. 4.2). Das heißt, dass keine weitere Differenzierung der Stromverbrauchsdaten erforderlich ist, und die Daten für die weitere Modellbildung im ARIMA-Modell verwendet werden können. Daher kann der Parameter d im ARIMA-Modell auf null gesetzt werden (keine Differenzierung nötig). Ein ARIMA-Modell mit

d=0 entspricht einem ARMA-Modell, da das I(d)-Element in der Modellierung keine Rolle spielt.

ADF Test Statistic : -10.19405328830838

p-value : 6.213140292067e-18

Abb. 4.2: ADF-Test Ergebnisse zur Bestimmung der Stationarität

Es wurde festgestellt, dass die Zeitreihendaten stationär sind. Daher können nun die weiteren Parameter p und q bestimmt werden. Um diese beiden Parameter festzulegen, werden ACF und PACF angewendet. Mit Statsmodels lassen sich ACF und PACF visualisieren, was die Analyse ermöglicht, wie stark die gegenwärtigen Werte einer Zeitreihe durch ihre historischen Werte beeinflusst werden und welche Verzögerungen tatsächlich signifikanten Einfluss auf die Zeitreihe haben (siehe Abb. 4.3).

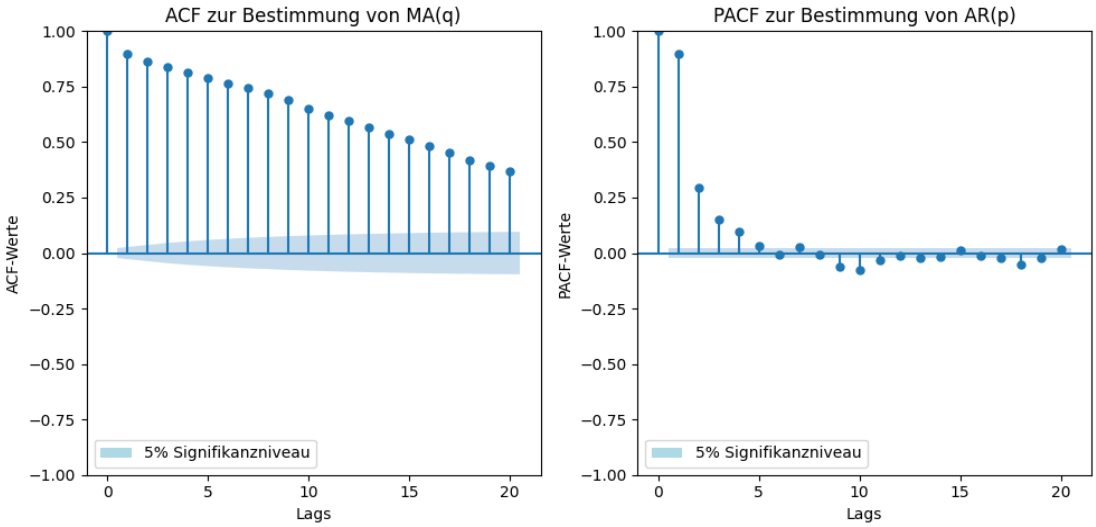

Abb. 4.3: ACF- und PACF-Diagramme zur Ermittlung von p und q

In Abb. 4.3 ist zu erkennen, dass die ACF-Werte sehr langsam absinken. Dies zeigt, dass zwischen den auseinanderliegenden Beobachtungen der Zeitreihe eine anhaltende Korrelation besteht. Die historischen Werte haben einen langfristigen, jedoch allmählich abnehmenden Einfluss auf die gegenwärtigen Werte. Aus diesem Grund ist ein MA-Modell für die Modellierung in dieser Arbeit nicht optimal geeignet. Im PACF-Plot ist ersichtlich, dass die PACF-Werte ab dem 6. Lag innerhalb des 5% Signifikanzniveaus, welches die Nullhypothese der Nicht-Korrelation testet, nahezu auf null absinken. Dies weist darauf hin, dass sie statistisch nicht signifikant sind. Das 5% Signifikanzniveau wird genutzt, um zu bestimmen, ob die gemessenen Korrelationen über die Erklärung durch Zufall hinausgehen. Liegt ein PACF-Wert

innerhalb dieses Niveaus nahe Null, bedeutet das, dass es keine überzeugende statistische Evidenz dafür gibt, dass dieser Lag einen wesentlichen Einfluss auf die Zeitreihe hat. Daher haben nur die ersten fünf Lags einen statistisch signifikanten und direkten Einfluss auf die aktuellen Werte der Zeitreihe. Im Gegensatz dazu tragen die weiteren Lags (nach dem fünften) nicht signifikant zur Vorhersage bei. Daher wird der Parameter p auf 5 gesetzt und q auf null reduziert, was zu einem AR(5)-Modell führt. Schlussendlich erhält man die Parameter (5,0,0) für das ARIMA-Modell, das einem AR(5)-Modell entspricht, da weder das MA-Modell noch das I-Modell zu einer signifikanten Verbesserung der Modellierung des Stromverbrauchs beitragen [32].

4.1.2 Schritte zur Modellierung

Nach der Festlegung der Parameter für das ARIMA(5,0,0)-Modell, auch als AR(5)-Modell bezeichnet, können die Zeitreihendaten und Eingabeparameter in das Modell integriert werden. Um Vorhersagen erstellen zu können, muss das ARIMA-Modell zunächst mithilfe der TimeSeriesSplit-Methode als Cross-Validation mit 10 Splits die Trainingsdaten erlernen. Dabei muss dem ARIMA-Modell mitgeteilt werden, welche die Eingabeparameter (exogene Variablen) und welches die Zielvariable (endogene Variable) sind (vgl. Abb. 3.12). Zum Trainieren des Modells wird die Methode *fit()* angewendet, um den Trainingsprozess zu starten. Anschließend kann mithilfe der Methode *forecast()* eine Vorhersage für die Validierungsdaten erstellt werden, um die Trainingsprozesse zu überwachen und die Metriken zu sammeln, mit denen die Modellleistung bewertet wird. Schließlich kann nach Abschluss des gesamten Trainings bei der Cross-Validation die Methode *forecast()* genutzt werden, um Prognosen für die Testdaten zu erstellen, die dann die Vorhersagegenauigkeit beurteilen (siehe Abb. 4.4).

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.model_selection import TimeSeriesSplit

# Cross-Validation (cv)
for train_index, val_index in TimeSeriesSplit(n_splits=10).split(train_data):
    cv_train, cv_val = train_data.iloc[train_index], train_data.iloc[val_index]

    # Modelltraining
    arima_model = SARIMAX(endog=cv_train[TARGET],
                          exog=cv_train[FEATURES],
                          order=(5, 0, 0)).fit()

    # Vorhersagen zu Validierungsdaten machen
    cv_pred = arima_model.forecast(steps=len(cv_val), exog=cv_val[FEATURES])

# Vorhersagen zu Testdaten machen
test_pred = arima_model.forecast(steps=len(test_data), exog=test_data[FEATURES])
```

Abb. 4.4: Programmcode zur Implementierung eines ARIMA-Modells

4.1.3 ARIMA-Prognosen zum Stromverbrauch

Der vorhergesagte Stromverbrauch vom 18. Oktober bis zum 21. November, der für das ARIMA-Modell bisher unbekannt war, bietet die Möglichkeit, einen Einblick in den zukünftigen Stromverbrauch zu erhalten (siehe Abb. 4.5).

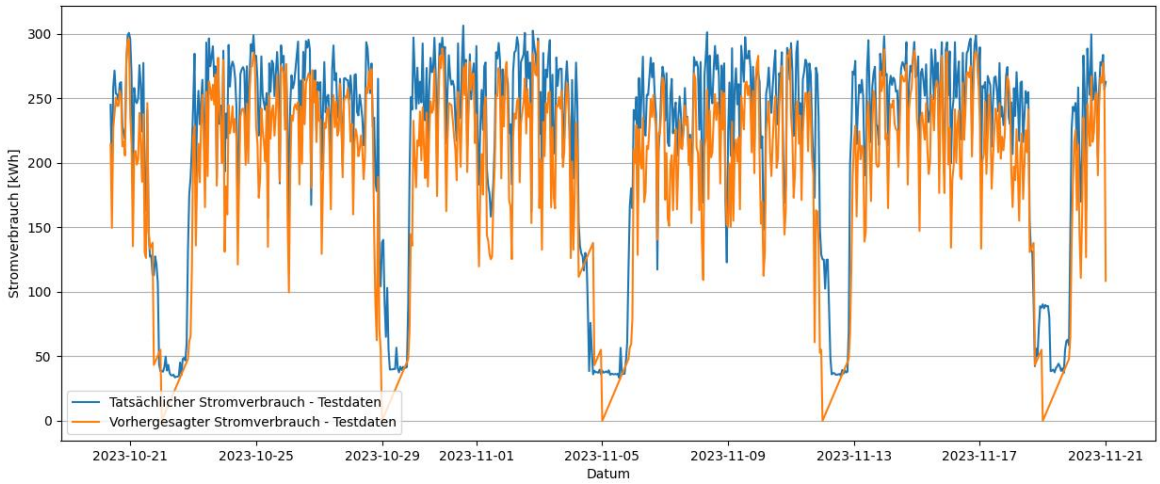

Abb. 4.5: Darstellung der Stromverbrauchsprognose durch ARIMA

In Abb. 4.5 zeigt sich, dass das ARIMA-Modell im Allgemeinen in der Lage ist, die Schwankungen des Stromverbrauchs zwischen normalen Arbeitstagen und Sonntagen zu beschreiben, was durch die Produktionsplanung (Kastenanzahl) unterstützt wurde. Jedoch wird der Stromverbrauch an Sonntagen unterschätzt, obwohl der tatsächliche Energieverbrauch eigentlich höher ist. Durch die Subtraktion des vorhergesagten Werts vom tatsächlichen Wert erhält man die Residuen. Gemäß der Analyse des Residuenplots neigt das ARIMA-Modell in dieser Arbeit dazu, die Schwankungen des Stromverbrauchs sowohl an Arbeitstagen als auch an Sonntagen zu unterschätzen (positive Residuen), wenn der tatsächliche Verbrauch nicht so niedrig ist (siehe Abb. 4.6).

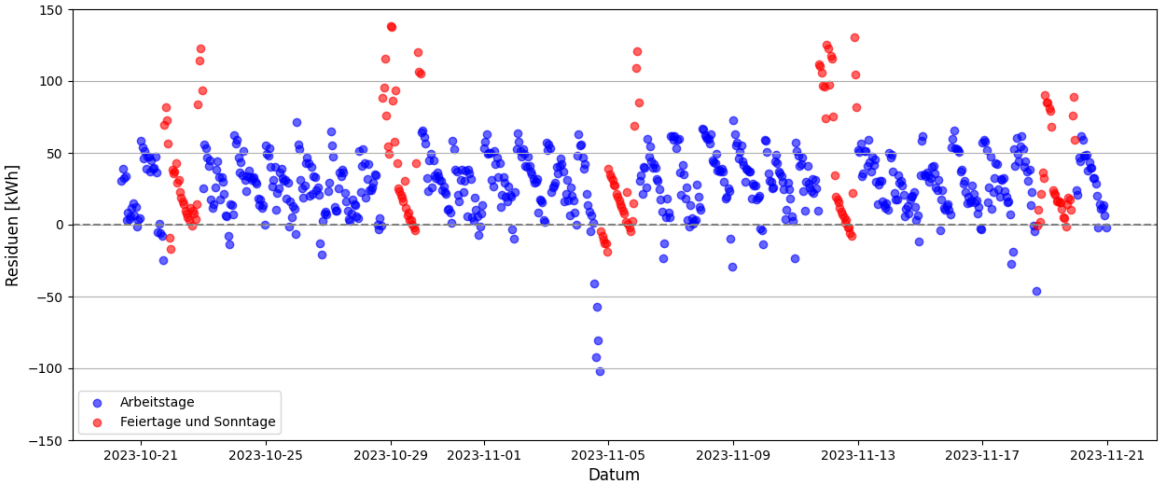

Abb. 4.6: Residualplot der ARIMA-Vorhersage für den Stromverbrauch

4.2 LSTM

In dieser Arbeit ist es nicht erforderlich, den komplexen Gating-Mechanismus von Grund auf neu zu erstellen. Stattdessen ermöglicht PyTorch die Nutzung der bereits implementierten LSTM-Architektur sowie des DL-Netzwerks für die Modellierung des Stromverbrauchs. Es gibt jedoch einige Hyperparameter, die den Lernprozess steuern und die Modellleistung weiter verbessern können. Die in der Implementierung des LSTM regulierten Hyperparameter werden im nächsten Unterkapitel erklärt und festgelegt.

4.2.1 Auswahl der Hyperparameter

Obwohl die LSTM-Architektur und das dazugehörige DL-Netzwerk vorgefertigt sind, erfordert der Einsatz dennoch die Festlegung einiger initialer Hyperparameter für das allgemeine Gerüst des LSTM-Netzwerks. Dazu gehört die Bestimmung der Zellanzahl im Input-Layer, in den Hidden-Layers und im Output-Layer sowie die Anzahl der Hidden-Layers. Dabei werden vier Neuronen im Input-Layer und ein Neuron im Output-Layer verwendet, um die Eingabeparameter und Zielvariablen anzupassen (vgl. Abb. 3.12). Die Anzahl der Hidden-Layers wird aufgrund begrenzter Rechenleistung auf 2 beschränkt, was als Minimum für das DL-Netzwerk gilt. Die optimale Anzahl von Neuronen in einem Hidden-Layer wird durch wiederholte experimentelle Versuche ermittelt, die die besten MAE- und R2-Werte liefern sollen. Laut dem Experiment erweist sich eine Anzahl von 16 Neuronen als ideal, daher wird diese Anzahl für den Hidden-Layer festgelegt (siehe Abb. 4.7).

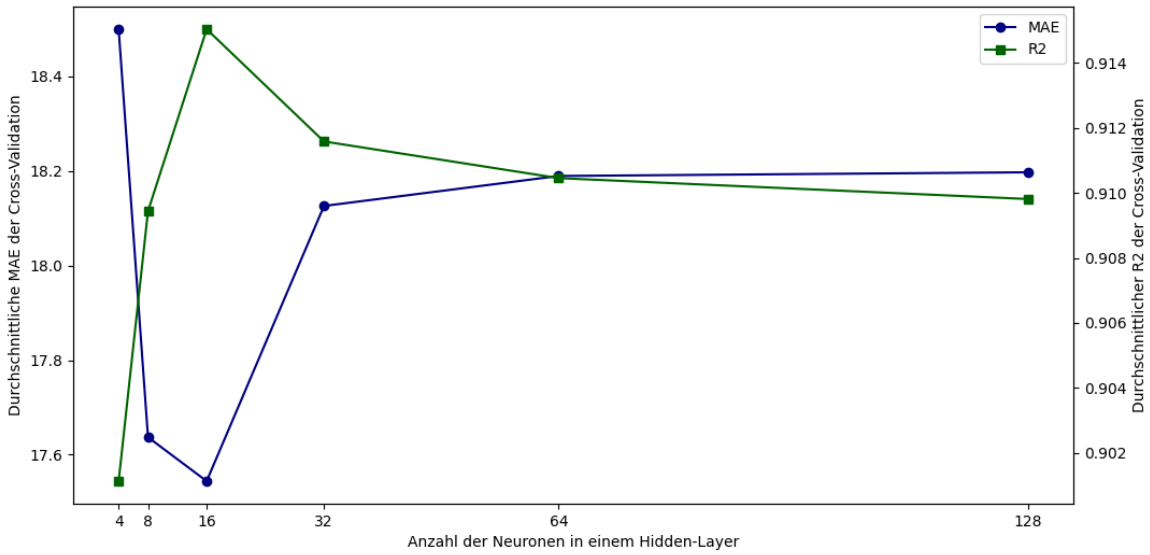

Abb. 4.7: LSTM-Leistung bei unterschiedlicher Anzahl an Neuronen. Im weiteren Verlauf wurden 16 Neuronen verwendet.

Im gesamten Lernprozess spielt die Lernrate als Hyperparameter eine wesentliche Rolle, da sie die Anpassungsgeschwindigkeit der Abweichungen zwischen

geschätzten und tatsächlichen Werten steuert. Um die ideale Lernrate zu finden, wird das LSTM-Modell innerhalb der Bereiche von 0,01 bis 0,1 bei gleicher Netzwerkstruktur trainiert (siehe Abb. 4.8).

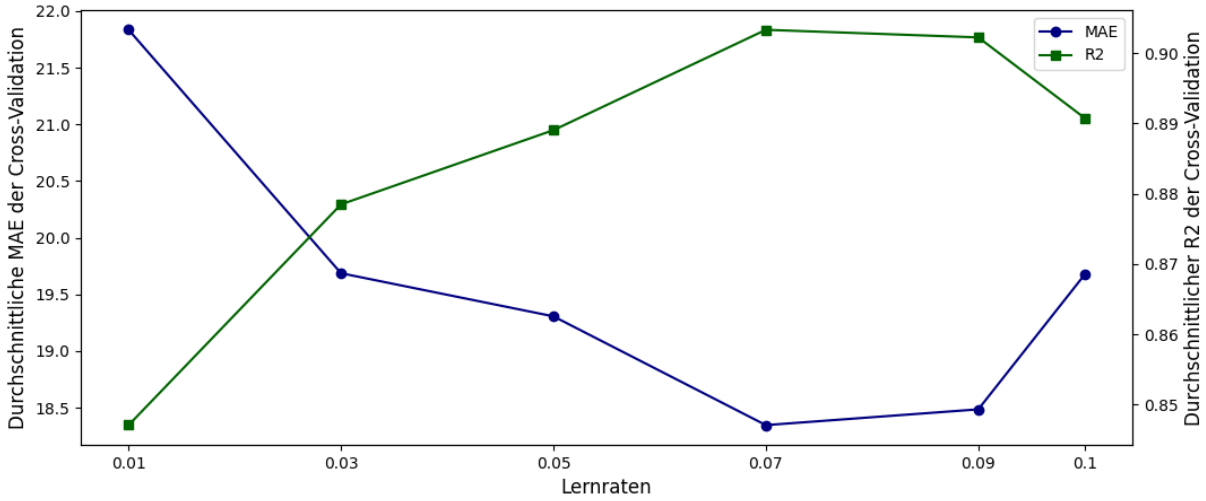

Abb. 4.8: LSTM-Leistung bei verschiedenen Lernraten. Im weiteren Verlauf wurde eine Lernrate von 0.07 verwendet.

In Abb. 4.8 ist ersichtlich, dass das Modell bei einer Lernrate von 0,07 die niedrigste MAE und einen höheren R2 aufweist, als alle anderen getesteten Lernraten. Aus diesem Grund wird 0,07 als Lernrate gewählt. Um das Lernverfahren und die Generalisierung weiter zu verbessern, ist die Anzahl der Epochen ein weiterer wichtiger Hyperparameter für das Training. Sie bestimmt, wie oft das Netzwerk dieselben Trainingsdaten betrachtet und daraus lernt. Eine übermäßige Anzahl von Epochen kann jedoch zu Overfitting führen, da das Netzwerk versucht, von identischen Daten zu häufig und zu intensiv zu lernen. Somit kann die Vorhersagegenauigkeit und die Generalisierung beeinträchtigt werden.

Daher kommt in dieser Arbeit die Funktion *EarlyStopping* zum Einsatz, die den Lernprozess automatisch überwacht und die besten Trainingsergebnisse identifiziert. Diese Funktion ermöglicht es, das Training innerhalb einer Cross-Validation automatisch zu stoppen, sobald die besten Evaluierungsergebnisse erreicht sind. Hierfür muss lediglich festgelegt werden, wie viele Versuchsrunden das Netzwerk nach Erreichen des besten Ergebnisses für weitere Verbesserungen noch durchführen darf, sowie die maximale Anzahl an Epochen, die den Trainingsprozess begrenzt, sodass das Netzwerk nicht unendlich weiterlernt. Es werden 10 Epochen festgelegt, was mit der Anzahl der Splits in der Cross-Validation übereinstimmt. Die Versuchsrunden werden auf 5 begrenzt, was der Hälfte der maximalen Epochenanzahl entspricht.

Zudem ist die gleichzeitige Verwendung aller Trainingsdaten für das Training eines LSTM- oder DL-Netzwerks aufgrund des erheblichen Speicherplatzbedarfs (zahlreiche

Neuronen und vernetzte Layers) und der hohen Rechenleistungsanforderungen nicht optimal. Das Netzwerk muss nämlich alle Trainingsdaten auf einmal verarbeiten. Daher wird in dieser Arbeit das Mini-Batch-Verfahren eingesetzt, welches die Trainingsdaten in kleinere Teile untergliedert. Auf diese Weise lernt das LSTM-Netzwerk mit weniger Daten, benötigt jedoch mehr Iterationen, was weniger speicherintensiv ist. Die optimale Batch-Größe wird ebenfalls durch zehn wiederholte Versuche bestimmt, wobei die Netzwerkstruktur und die Lernrate identisch bleiben. In Abb. 4.9 zeigt sich, dass ein Modell mit einer Batch-Größe von 32 die niedrigste MAE und den höchsten R2 bei der Validierung erreicht. Daher wird die Batch-Größe auf 32 festgelegt.

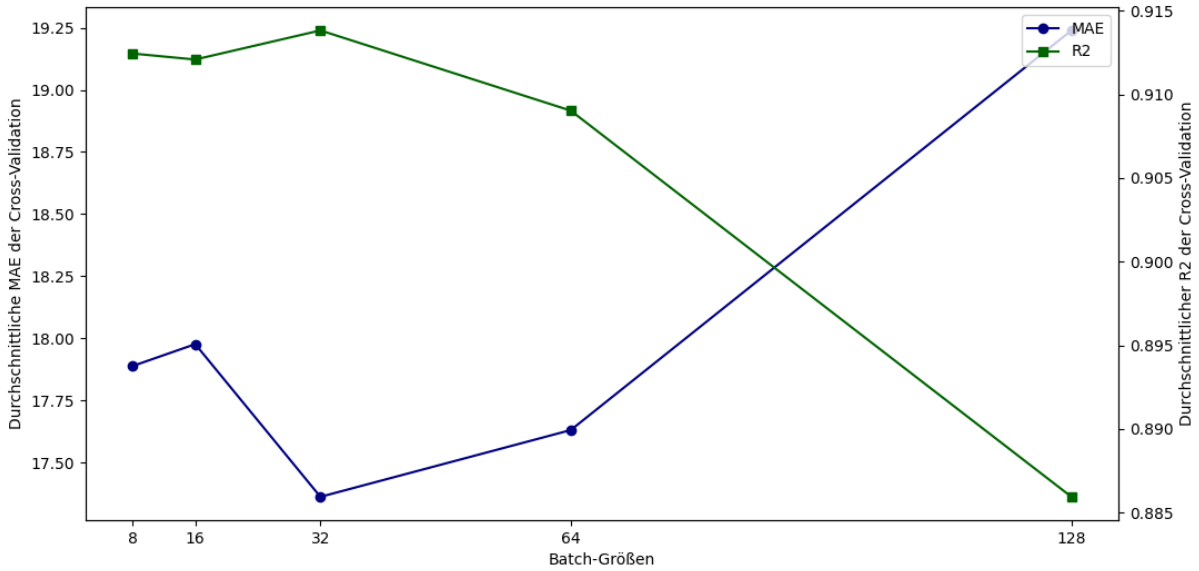

Abb. 4.9: LSTM-Leistung bei verschiedenen Batch-Größen. Im weiteren Verlauf wurde eine Batch-Größe von 32 verwendet.

Ein weiterer Vorteil des LSTM-Modells liegt in der Bewahrung von Informationen über vergangene Datenpunkte, um den gegenwärtigen Stromverbrauch anhand aktueller und vergangener Eingabeparameter zu schätzen. In diesem Zusammenhang wird die Verzögerung (Lag) als Hyperparameter betrachtet, der zur weiteren Optimierung der Modellleistung angepasst werden kann. Die Menge der Lags bestimmt, wie weit das LSTM-Modell in die Vergangenheit zurückblicken muss, um für die aktuelle Zielvariable zu lernen. Um einen Überblick über die Modellleistung bei verschiedenen Lags zu erhalten, werden zehn wiederholte Versuche durchgeführt, die vom letzten bis zum achten Lag innerhalb einer Schicht reichen (insgesamt 8 Stunden). Laut dem Experiment bietet das Modell mit 5 Lags die niedrigste MAE und einen hohen R2-Wert. Aus diesem Grund werden die Lags auf 5 eingestellt, sodass das LSTM-Modell aus den vergangenen 5 Datenpunkten (5 Stunden) lernt und somit die Vorhersage trifft. Dies deckt sich mit den Ergebnissen der PACF-Analyse im Rahmen der ARIMA-Modellierung (siehe Abb. 4.10).

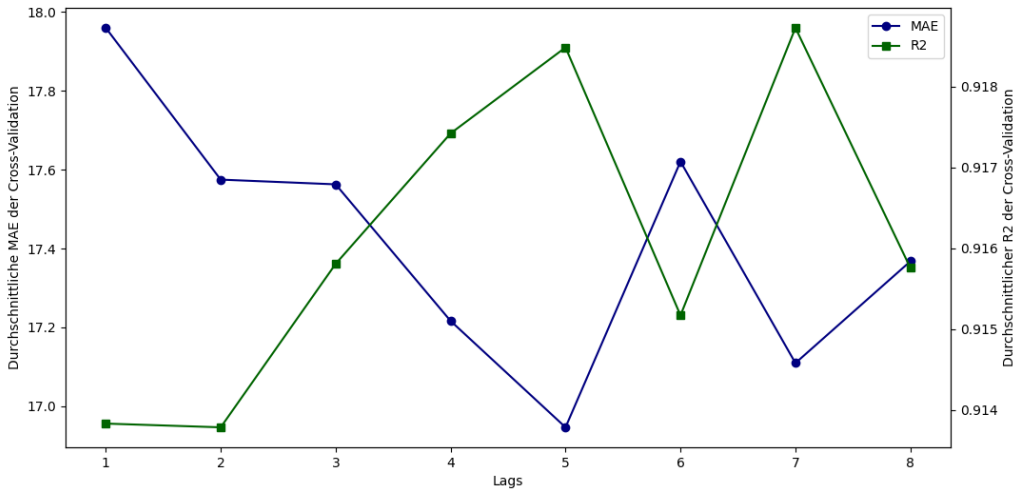

Abb. 4.10: LSTM-Leistung bei verschiedenen Lags. Im weiteren Verlauf wurde ein maximaler Lag von 5 verwendet.

4.2.2 Schritte zur Modellierung

Nach der Festlegung der Hyperparameter können die Zeitreihendaten und Eingabeparameter nun in das LSTM-Modell eingespeist werden. Beginnend mit der Datenskalierung, die mit der Funktion *MinMaxScaler* durchgeführt wird, um die Features und Zielvariablen auf den Bereich von -1 bis 1 zu normieren, wird der Trainingsprozess beschleunigt und die numerische Stabilität verbessert. Im Gegensatz dazu wurden die Daten für das ARIMA- und XGBoost-Modell nicht skaliert, um die Interpretierbarkeit zu bewahren. Eine Untersuchung der Skalierung wurde durchgeführt, zeigte jedoch keine Verbesserung der Modellleistung.

Nach der Skalierung werden die Daten in NumPy-Arrays transformiert und anschließend in eine für das Training eines LSTM-Netzwerks geeignete Form von PyTorch-Tensoren überführt. Für das Training wird ein LSTM-Modell mit den in Kapitel 4.2.1 festgelegten Hyperparametern initialisiert. Dabei erlernt das LSTM-Modell mithilfe der TimeSeriesSplit-Methode als Cross-Validation mit 10 Splits die Trainingsdaten, ähnlich wie beim ARIMA-Modell (vgl. Abb. 4.4). Schließlich werden die Testdaten in Form von PyTorch-Tensoren umgewandelt, um Vorhersagen zu generieren. Diese Vorhersagen basieren auf dem gelernten Modell und zeigen, wie gut das LSTM zeitabhängige Muster in den Daten erkennen kann. Dieser gesamte Prozess bildet eine umfassende Methode, um mit DL Zeitreihenprognosen durchzuführen und deren Leistung zu bewerten (siehe Anhang A1).

4.2.3 LSTM-Prognosen zum Stromverbrauch

Die Vorhersage des Stromverbrauchs vom 18. Oktober bis zum 21. November, derselbe Zeitraum wie beim ARIMA-Modell, die durch das LSTM-Modell ermöglicht wird, gibt die Möglichkeit, Einblicke in den zukünftigen Energiebedarf zu erhalten (siehe Abb. 4.11).

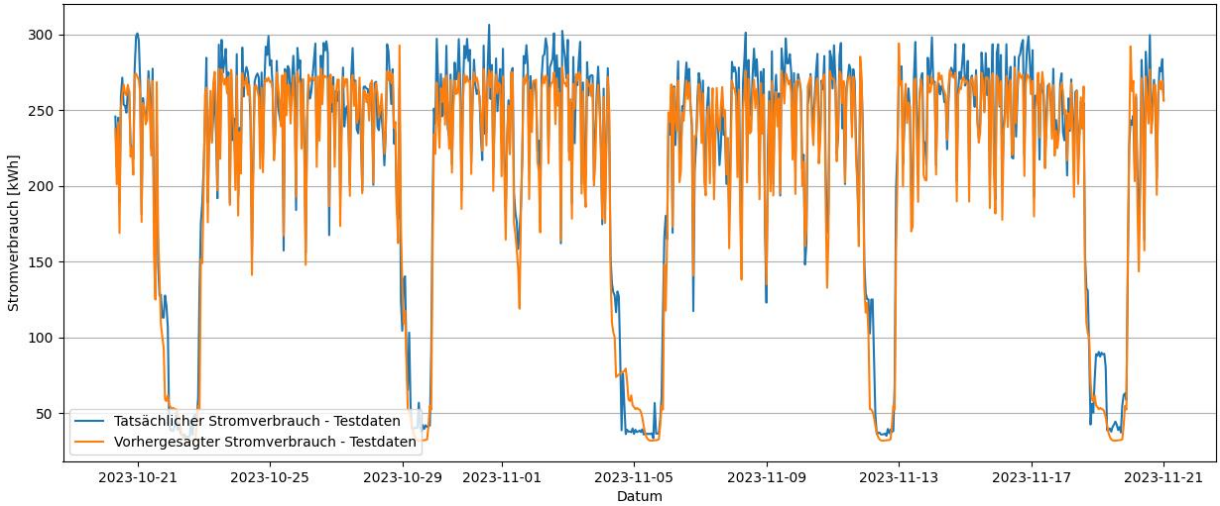

Abb. 4.11: Darstellung der Stromverbrauchsprognose durch LSTM

In Abb. 4.11 kann man sehen, dass das LSTM-Modell die Schwankungen an gewöhnlichen Arbeitstagen und Sonntagen genauer nachbilden kann als das ARIMA-Modell (vgl. Abb. 4.5). Zudem liefert das LSTM-Modell eine deutlich bessere Schätzung des Stromverbrauchs an Sonntagen, während das ARIMA-Modell den Stromverbrauch tendenziell unterschätzt. Die Untersuchung des Residuenplots zeigt, dass die meisten Residuen des LSTM-Modells im Bereich zwischen -50 und 50 liegen. Dabei ist zu erkennen, dass das LSTM-Modell im Vergleich zum ARIMA-Modell einen kleineren Residualbereich aufweist. Dies bedeutet, dass die Abweichungen zwischen den tatsächlichen Werten und den durch LSTM prognostizierten Werten geringer sind als beim ARIMA-Modell. Insgesamt bietet das LSTM-Modell eine treffendere Darstellung der Stromschwankungen im Vergleich zum ARIMA-Modell (siehe Abb. 4.12).

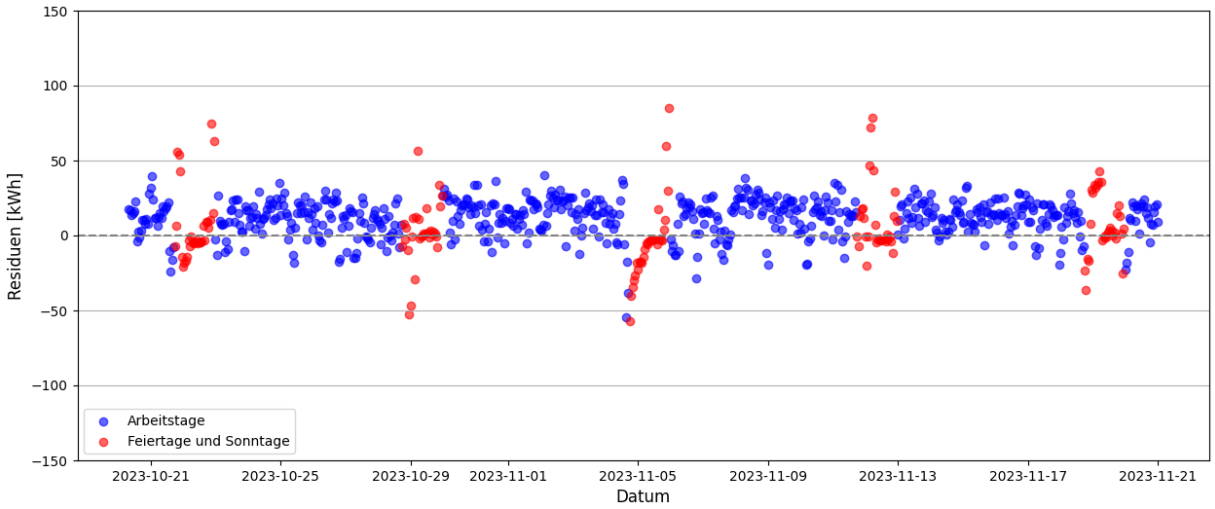

Abb. 4.12: Residualplot der LSTM-Vorhersage für den Stromverbrauch

4.3 XGBoost

Für die Modellbildung wird das Python-Paket XGBoost genutzt, das den Einsatz des gleichnamigen Entscheidungsbaum-Boosting-Algorithmus ermöglicht. Allerdings verfügt XGBoost über einige Hyperparameter, mit denen die Modellleistung verbessert werden kann. Die Anpassung dieser Hyperparameter beim Entwickeln des XGBoost-Modells wird im nächsten Unterkapitel erläutert und festgelegt.

4.3.1 Auswahl der Hyperparameter

Wie beim LSTM-Modell hat auch XGBoost eine Hyperparameter Lernrate, die die Geschwindigkeit der Modellupdates steuert. Um die optimale Lernrate zu ermitteln, wird ein Experiment durchgeführt, bei dem das XGBoost-Modell mit verschiedenen Lernraten zwischen den typischen Bereichen von 0,1 bis 1 trainiert und anschließend mit MAE und R2 evaluiert wird. Die Ergebnisse zeigen, dass eine Verbesserung der Modellleistung bei einer Lernrate über 0,3 nicht mehr signifikant ist. Aus diesem Grund wird die Lernrate von 0,3 beibehalten, welche standardmäßig im XGBoost-Paket verwendet wird (siehe Abb. 4.13).

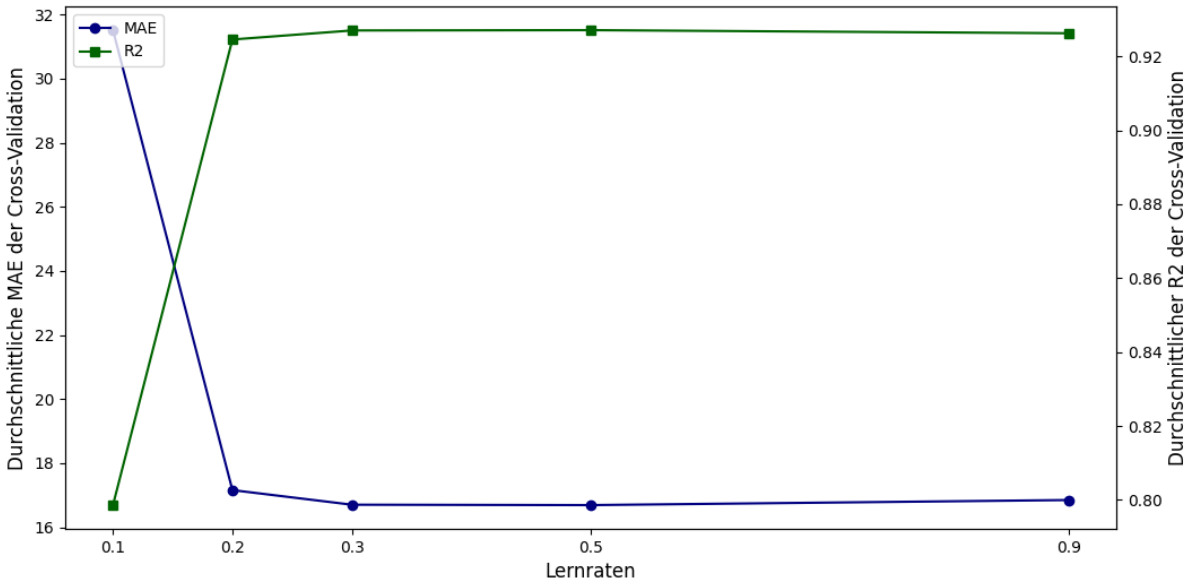

Abb. 4.13: XGBoost-Leistung bei verschiedenen Lernraten. Im weiteren Verlauf wurde eine Lernrate von 0.3 verwendet.

Für eine weitere Verbesserung der Modellleistung kommt der Estimator zum Einsatz, der die Anzahl der zu erstellenden Entscheidungsbäume angibt. Da das grundlegende Konzept des XGBoost-Algorithmus auf der Kombination von Entscheidungsbäumen beruht, ist dieser Hyperparameter für die Umsetzung des XGBoost-Modells bedeutend. Hierzu wird ein Test durchgeführt, um einen Überblick über die Modellleistung mit einer unterschiedlichen Anzahl an Estimatoren zu erhalten und um festzulegen, wie viele Entscheidungsbäume erstellt werden sollen (siehe Abb. 4.14).

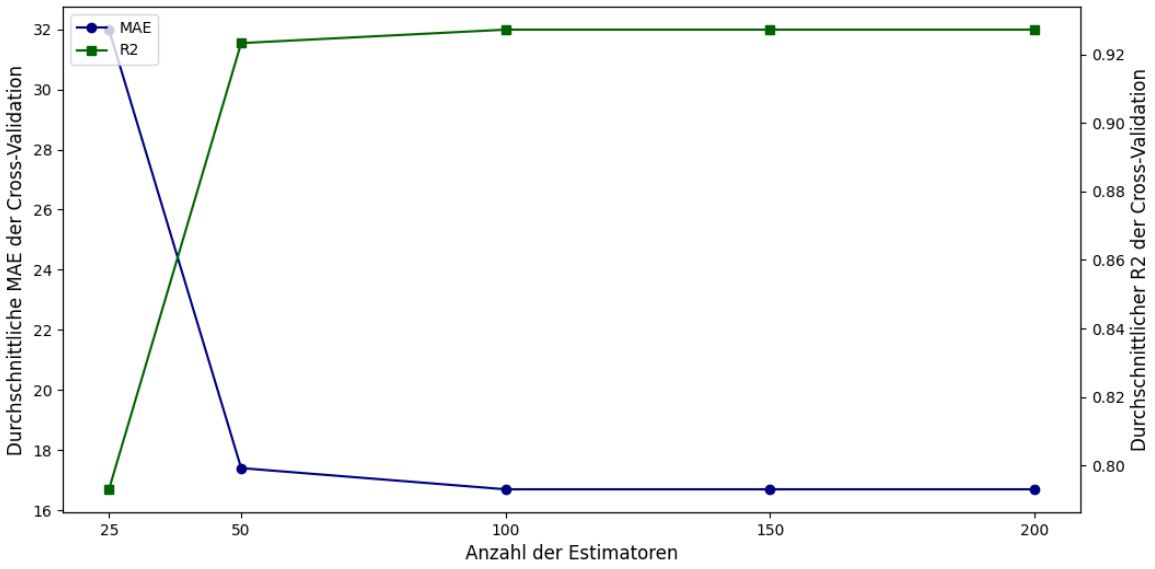

Abb. 4.14: XGBoost-Leistung bei verschiedenen Estimatoren. Im weiteren Verlauf wurden 100 Estimatoren verwendet.

In Abb. 4.14 ist der Einfluss einer unterschiedlichen Anzahl an erstellten Entscheidungsbäumen zu sehen. Deutlich erkennbar ist, dass mehr als 100 Bäume die Modellleistung nicht weiter signifikant verbessern können. Daher wurden hier nur 100 Estimatoren verwendet, um die Verschwendung von Rechenressourcen zu vermeiden.

4.3.2 Schritte zur Modellierung

Nachdem die Hyperparameter festgelegt wurden, können nun die Zeitreihendaten und Eingabeparameter in das XGBoost-Modell eingegeben werden. Um Vorhersagen zu ermöglichen, muss das Modell zunächst mit der TimeSeriesSplit-Methode als Cross-Validation mit 10 Splits trainiert werden. Hierbei ist es wichtig, im Modell die Eingabeparameter (Features) und die Zielvariable zu definieren (vgl. Abb. 3.12). Zur Einleitung des Trainingsprozesses wird die Methode *fit()* verwendet. Anschließend erzeugt das Modell mit der Methode *predict()* Vorhersagen für die Validierungsdaten, welche zur Überwachung der Trainingsprozesse und zur Messung der Modellleistung dienen. Nach Abschluss der Cross-Validation bzw. des Trainings wird die *predict()*-Methode genutzt, um Prognosen für die Testdaten zu erstellen und so die Vorhersagegenauigkeit zu bewerten (siehe Abb. 4.15).

```
import xgboost as xgb
from sklearn.model_selection import TimeSeriesSplit

# Cross-Validation (cv)
for train_index, val_index in TimeSeriesSplit(n_splits=10).split(train_data):
    cv_train, cv_val = train_data.iloc[train_index], train_data.iloc[val_index]

    # Modelltraining
    xgb_model = xgb.XGBRegressor(n_estimators=100,
                                 lr=0.3).fit(cv_train[FEATURES], cv_train[TARGET])

    # Vorhersagen zu Validierungsdaten machen
    cv_pred = xgb_model.predict(cv_val[FEATURES])

# Vorhersagen zu Testdaten machen
test_pred = xgb_model.predict(test_data[FEATURES])
```

Abb. 4.15: Programmcode zur Implementierung eines XGBoost-Modells

4.3.3 XGBoost-Prognosen zum Stromverbrauch

Die Vorhersage des Stromverbrauchs vom 18. Oktober bis zum 21. November, derselbe Zeitraum wie beim ARIMA- und LSTM-Modell, die durch das XGBoost-Modell erstellt wird, ermöglicht Einblicke in den zukünftigen Energiebedarf in der Produktion (siehe Abb. 4.16).

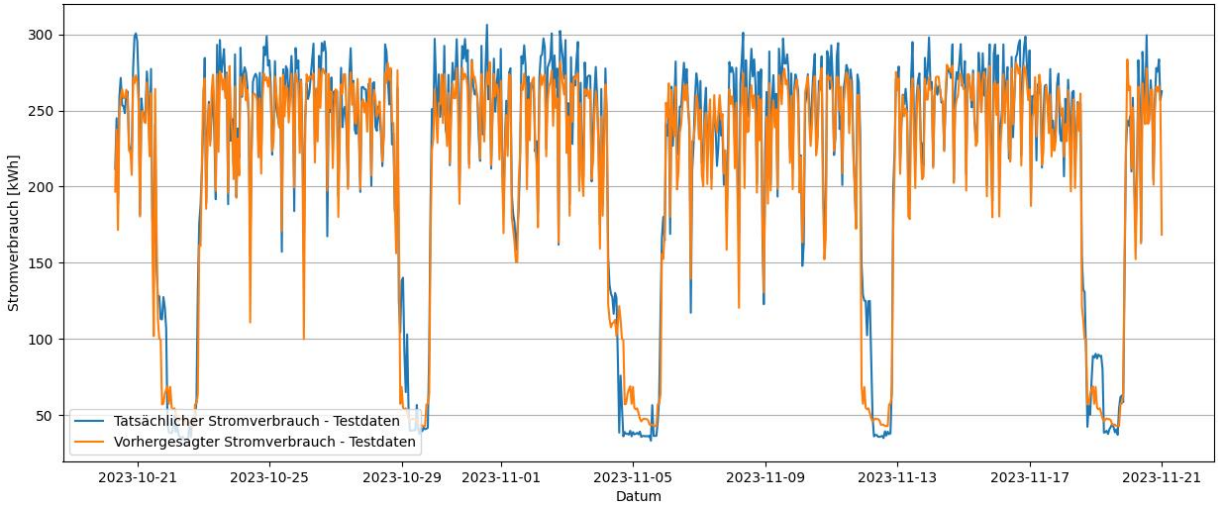

Abb. 4.16: Darstellung der Stromverbrauchsprognose durch XGBoost

Generell schätzt das XGBoost-Modell den künftigen Stromverbrauchsverlauf ähnlich wie das LSTM-Modell, was die Beurteilung der Modellleistung erschwert. In Abb. 4.16 lässt sich erkennen, dass das XGBoost-Modell die Schwankungen an Arbeitstagen ähnlich präzise nachbilden kann wie das LSTM-Modell. Allerdings schätzt das XGBoost-Modell den Stromverbrauch an manchen Sonntagen ungenauer als das LSTM-Modell. Im Residuenplot kann man sehen, dass die meisten Residuen des XGBoost-Modells an Arbeitstagen im Bereich zwischen -50 und 50 liegen, was vergleichbar mit dem LSTM-Modell ist. Hingegen hat das XGBoost-Modell an manchen Sonntagen größere Residuen als das LSTM-Modell, die sich im Bereich zwischen 50 und 100 befinden (siehe Abb. 4.17).

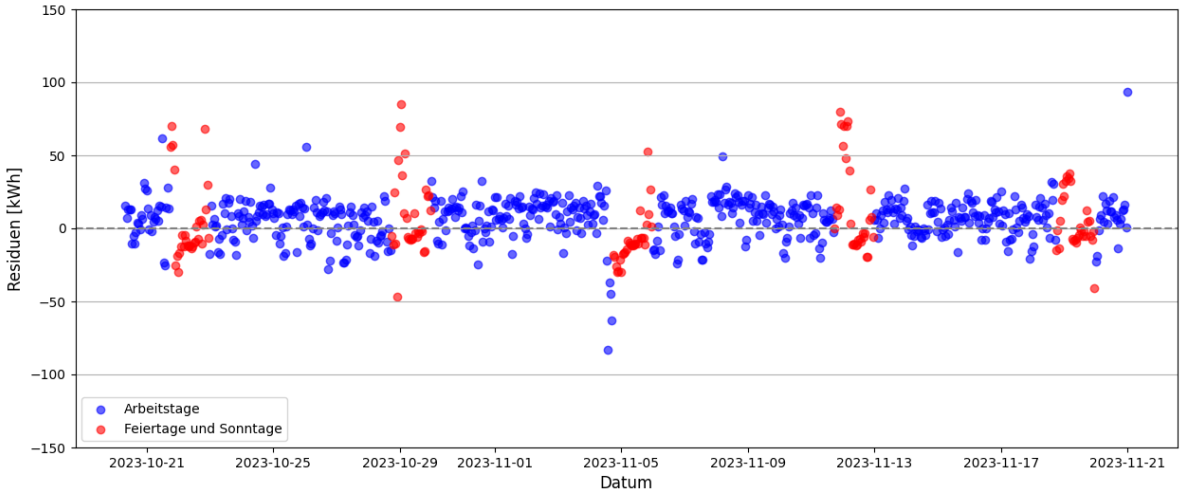

Abb. 4.17: Residualplot der XGBoost-Vorhersage für den Stromverbrauch

Allerdings lässt sich durch die Visualisierung alleine noch nicht beurteilen, welches Modell bessere Ergebnisse liefert. Daher werden die im Trainingsprozess durch Cross-Validation gesammelten Evaluierungsmetriken (MAE, RMSE und R2) im nächsten Kapitel verwendet, um die Genauigkeit der drei Modellierungen objektiver zu vergleichen. Zudem kommen Evaluierungsmetriken zur Anwendung, die auf den Testdaten basieren.

5 Auswertung der Modelle

Dieses Kapitel evaluiert und vergleicht die Leistung sowie die Vorhersagegenauigkeit der Modelle basierend auf den Metriken, die in Kapitel 2.4.2 beschrieben wurden. Es wird untersucht, welches Modell die besten Ergebnisse in der Modellierung und Vorhersage des Stromverbrauchs erzielt, welches die geringste Trainingszeit erfordert und welches für den zukünftigen Einsatz am besten geeignet ist.

5.1 Leistungsvergleich der Vorhersagemodelle

Um die drei implementierten Vorhersagemodelle (ARIMA, LSTM und XGBoost) präziser beurteilen zu können, wird der quantifizierte Evaluierungsansatz angewendet. Die verwendeten Metriken umfassen MAE, RMSE und R2, die quantifizierte Ergebnisse liefern können. Hier werden die durchschnittlichen Metriken von 10 wiederholten Validierungen während der Cross-Validation genutzt, um die allgemeine Modellleistung zu vergleichen. Des Weiteren werden die aus den Testdaten gesammelten Metriken für die Bewertung der Vorhersagegenauigkeit genutzt, welche repräsentieren, wie gut das Modell die unbekannte Zukunft einschätzen kann.

In Tab. 5.1 ist zu sehen, dass das ARIMA-Modell die höchsten Werte für MAE und RMSE sowie die niedrigsten Werte für R2 in den Validierungs- und Testsätzen erreicht. Dies deutet darauf hin, dass das ARIMA-Modell den Stromverbrauch im Vergleich zu den beiden anderen Modellen generell schlechter modelliert und prognostiziert. Für die Validierungssätze erzielt das XGBoost-Modell etwas bessere Ergebnisse als das LSTM-Modell, allerdings zeigt das LSTM-Modell in den Testsätzen eine etwas bessere Vorhersagegenauigkeit im Vergleich zu XGBoost.

Tab. 5.1: Leistungsvergleich von ARIMA, LSTM und XGBoost

Modelle	**Validierung (10 Splits)**			**Testdaten**		
	MAE	RMSE	R2	MAE	RMSE	R2
ARIMA	33,442	42,625	0,698	32,932	41,144	0,741
LSTM	18,073	23,424	0,915	**12,481**	**16,353**	**0,959**
XGBoost	**16,801**	**21,657**	**0,927**	13,436	18,032	0,950

Aus den Ergebnissen lässt sich noch keine Entscheidung treffen, ob XGBoost oder LSTM das optimale Modell für diese Arbeit ist. Die Bewertung der Validierungssätze ist theoretisch konservativer und objektiver als die der Testsätze, da die Modelle in diesem Fall zehnmal verifiziert werden, was eine gründlichere Überprüfung der durchschnittlichen Modellleistung ermöglicht. Im Gegensatz dazu werden die Modelle in den Testsätzen nur einmal getestet, was möglicherweise zu einer zufälligeren Bewertung führen könnte, weil die Vorhersagegenauigkeit nur von einem einzigen Test

abhängt. Um das beste Modell zu finden, wird im nächsten Unterkapitel eine tiefere Analyse bezüglich der Trainings- und Prognosedauer der Modelle durchgeführt.

5.2 Effizienzvergleich der Vorhersagemodelle

Beim Vergleich der Modellleistung und Vorhersagegenauigkeit ist der Unterschied zwischen LSTM und XGBoost nicht eindeutig. Daher werden auch die Zeiten berücksichtigt, die die drei Modelle für das Training und die Erstellung der Prognose benötigen. Tab. 5.2 zeigt, dass das XGBoost-Modell sowohl während des Trainings als auch bei der Vorhersage die geringste Zeit benötigt. Obwohl das XGBoost-Modell vergleichbare Leistung und Genauigkeit bei Vorhersagen wie das LSTM-Modell aufweist, trainiert es deutlich schneller (unter einer Sekunde). Im Vergleich zum ARIMA-Modell weist das LSTM-Modell eine kürzere Trainingszeit (ca. 35 Sekunden) und eine signifikant bessere Modellleistung auf.

Tab. 5.2: Effizienzvergleich von ARIMA, LSTM und XGBoost

Modelle	**Trainingsdauer (s)**	**Prognosedauer (s)**
ARIMA	69,679	0,029
LSTM	35,371	0,042
XGBoost	**0,520**	**0,003**

Nach dem Vergleich der Trainingsdauer lässt sich sagen, dass das XGBoost-Modell die kürzeste Trainingszeit und eine effizientere Modellierung sowie Vorhersage bietet als die anderen beiden Modelle. Weiterhin kann man feststellen, dass das LSTM-Modell die zweitkürzeste Trainingsdauer benötigt, während das ARIMA-Modell die längste Trainingszeit unter den drei Modellen in dieser Arbeit zeigt. Im nächsten Unterkapitel wird eine allgemeine Bewertung der drei Modelle abgegeben und geklärt, welches Modell für die potenzielle zukünftige Anwendung genutzt werden kann.

5.3 Zusammenfassung der Vergleichsergebnisse

Zusammenfassend kann gesagt werden, dass das ARIMA-Modell insgesamt die schlechteste Modellleistung und Vorhersagegenauigkeit aufweist sowie die ineffizienteste Modellierung (längste Trainingsdauer) hat. Aufgrund dieser Ergebnisse wird das ARIMA-Modell für zukünftige Vorhersageanwendungen in dieser Arbeit ausgeschlossen. Das LSTM-Modell und das XGBoost-Modell erzielen in den Validierungs- und Testsätzen sehr ähnliche Ergebnisse. Dabei liefert das XGBoost-Modell ein etwas besseres Ergebnis in der Cross-Validation, während das LSTM-Modell in den Testdaten etwas besser abschneidet. Allerdings weist das XGBoost-Modell eine deutlich kürzere Trainingszeit auf als das LSTM-Modell und verzeichnet die kürzeste Trainingsdauer unter den drei Modellen. Bei dem unentschiedenen Leistungsergebnis zwischen LSTM und XGBoost bietet das XGBoost-Modell einen effizienteren Lösungsansatz für die Modellierung des Stromverbrauchs. Mit diesem

kann auch auf leistungsbeschränkten Computern und Laptops eine genaue Prognose sowie Echtzeitanalyse realisiert werden. Daher wird das XGBoost-Modell aufgrund seiner Präzision und Effizienz bei der Modellierung und Vorhersage vorrangig empfohlen und ist für potenzielle zukünftige Anwendungen besser geeignet.

6 Anwendung

Der vorhergesagte Stromverbrauch hilft dabei, die damit verbundenen Stromkosten in der Produktion zu berechnen. Dadurch können fundiertere Entscheidungen getroffen werden, wie man die Produktion in Phasen mit günstigeren Strompreisen verlagert. Um die Simulation des Optimierungsmodells für Stromkosten zu vereinfachen und den Effekt anschaulich darzustellen, wird hier nur ein Arbeitstag (der 03. November 2023) aus den Testdaten ausgewählt. Die Strompreisdaten werden von der Website Energie-Charts bezogen, die in dieser Anwendung als Referenzwert dient [33]. Ohne die Berücksichtigung zusätzlicher Gebühren erhält man die gesamten Stromkosten pro Stunde, die die potenziellen Kosten in der Produktion darstellen. Die gesamten Stromkosten pro Stunde sind wie folgt definiert als:

$$K = V \times P \tag{6.1}$$

,mit

K = Gesamtkosten pro Stunde

V = Stündlicher Stromverbrauch in Kilowattstunden

P = Strompreis pro Kilowattstunde in Euro

Für das Beispiel zur Optimierung der Stromkosten wird das XGBoost-Modell aufgrund seiner Genauigkeit und des schnellen Trainings verwendet, um die Simulation der Preisschwankungen effizienter zu visualisieren. Zuerst kann man den von XGBoost prognostizierten stündlichen Stromverbrauch mit dem Strompreis pro Kilowattstunde multiplizieren, was die Gesamtkosten pro Stunde repräsentiert (siehe Abb. 6.1).

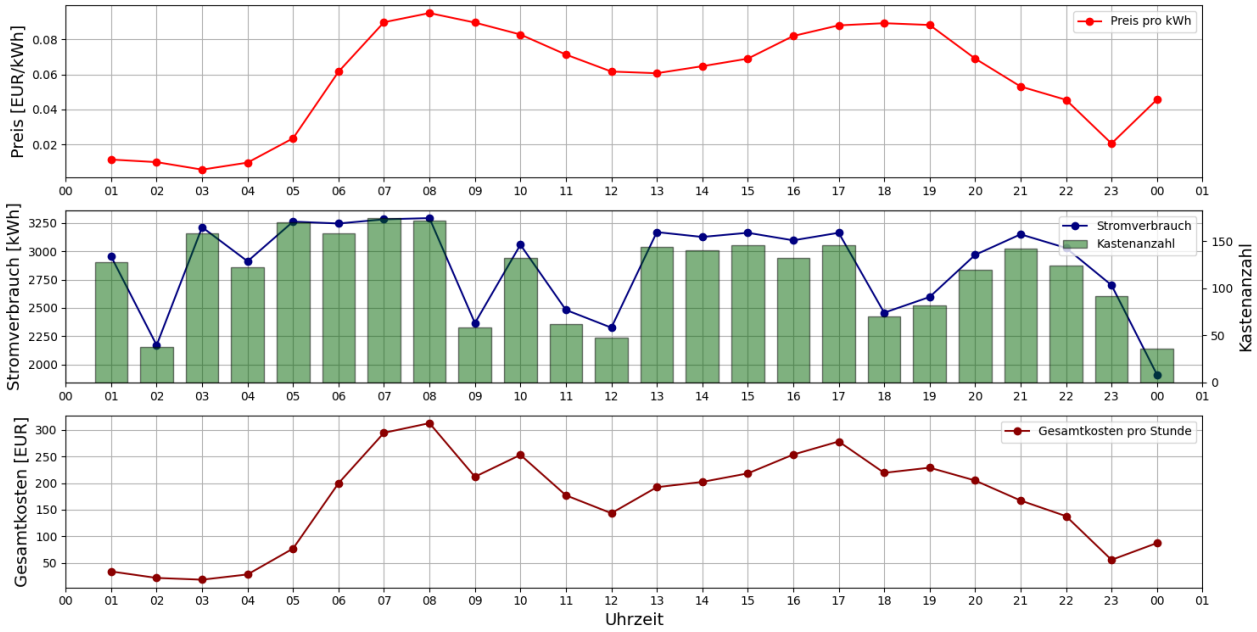

Abb. 6.1: Strompreis- und Verbrauchsdynamik, 03. November 2023

In Abb. 6.1 kann man sehen, dass die stündlichen Gesamtkosten zwei Spitzen aufweisen, die mit den Phasen der zwei teuersten Zeitperioden der Preislinie übereinstimmen. Diese Spitzen treten um 8 Uhr morgens und um 18 Uhr abends auf, Zeiten, zu denen die meisten Verbraucher Strom beziehen wollen. Nach der Zusammenrechnung von 24 Datenpunkten der Stromkosten belaufen sich die Gesamtkosten am 03. November auf 4019,52 EUR, bei einem Produktionsvolumen von 2793 Kästen. Dabei gibt es die Möglichkeit, die Herstellung der Kästen (Produkte) auf andere Zeitpunkte zu verschieben. Zum Beispiel sind zwischen 1 Uhr und 4 Uhr oder um 23 Uhr die Strompreise niedriger, wodurch man die Stromkosten reduzieren kann. Der Produktionsplaner berücksichtigte am 03. November weder den potenziellen Stromverbrauch noch die damit verbundenen Stromkosten, was zu einer Erhöhung der Produktionsaktivitäten führte, trotz außerordentlich hoher Strompreise wie zum Beispiel zwischen 7 Uhr und 8 Uhr.

Zudem lässt sich beobachten, dass die Kastenanzahl einen indirekten Einfluss auf die gesamten Stromkosten hat, da der Stromverbrauch, ein wesentlicher Faktor der Gesamtkosten, eine in Kapitel 3.4 analysierte hohe Korrelation mit der Kastenanzahl aufweist. Daher kann man die Kastenanzahl an die stündlichen Stromkosten anpassen, sodass keine Herstellung hoher Kastenzahlen zu Zeiten mit hohen Strompreisen geplant wird. Dadurch besteht um 3 Uhr eine signifikante Möglichkeit zur Steigerung der Produktion von Kästen, da die Strompreise zu dieser Zeit erheblich niedriger sind. Die korrekte maximale und minimale Fertigung der Kästen bei der Firma Fondium ist von deren technologischer Kapazität und Ressourcenverfügbarkeit abhängig. Für eine vereinfachte Simulation wird jedoch hier festgelegt, dass normalerweise stündlich maximal 170 Kästen hergestellt werden. Zudem sollten jede Stunde mindestens 50 Kästen produziert werden, um zu verhindern, dass die Produktion aufgrund hoher Strompreise stillsteht.

Für die Optimierung der Produktionsplanung wird ein Algorithmus entwickelt, der die Verteilung der Kastenanzahl innerhalb eines DataFrames anpasst. Dieser Algorithmus zielt darauf ab, die geplante Menge der Kästen (Produkte) während Zeiten hoher Strompreise zu verringern und dadurch die Gesamtkosten zu minimieren. Die höchsten und niedrigsten Preisspitzen werden ermittelt, wobei die Anzahl der Kästen in den teuersten Stunden auf mindestens 50 begrenzt wird. Die Kästen werden entsprechend den billigsten Stunden zugewiesen, bis das Maximum von 170 Kästen erreicht ist. Dieser Vorgang wird wiederholt, bis sich die Gesamtanzahl der Kästen nicht mehr verändert, was sicherstellt, dass das Gesamtvolumen erhalten bleibt (siehe Anhang A2).

Nach der Optimierung der Produktionsplanung bleibt die Gesamtzahl der zu produzierenden Kästen mit 2794 Stücken unverändert. Nun prognostiziert das trainierte XGBoost-Modell erneut den möglichen Stromverbrauch am 03. November, jedoch mit einer verbesserten Produktionsplanung. Dadurch erhält man einen

aktualisierten Überblick über die zukünftigen Stromkosten in der Produktion. In Abb. 6.2 ist zu sehen, dass die höchsten stündlichen Kosten von 300 EUR auf ungefähr 225 EUR reduziert wurden, weil hohe Produktionsaktivitäten während der teuersten Stunden vermieden werden. Die Gesamtkosten wurden optimiert und betragen nun 3710,55 EUR, was unter den ursprünglichen Kosten der ersten Planung (4019,52 EUR) liegt. Das Einsparpotenzial beträgt somit 308,97 EUR. Bei der optimierten Planung könnte die Firma 7,69 % der ursprünglichen Gesamtkosten sparen, was eine signifikante Kosteneffizienzsteigerung darstellt. Es muss jedoch berücksichtigt werden, dass dieser Wert nur das Maximum des möglichen Einsparpotentials darstellt, da der Börsenstrompreis als Grundlage diente. In der Praxis könnte das Einsparpotential möglicherweise nicht so hoch sein, da Verbraucher höhere Preise zahlen müssen.

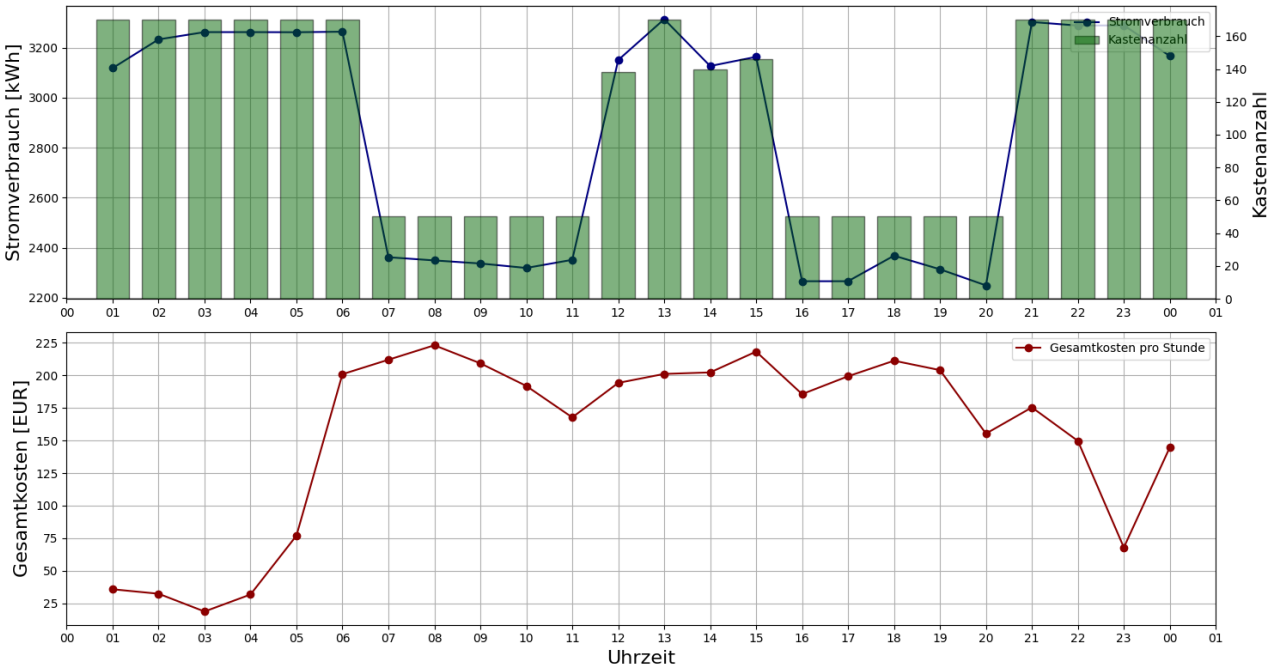

Abb. 6.2: Optimierung der Stromkosten durch Anpassung der Verbrauchsdynamik, 03. November 2023

Um einen Überblick über den Verlauf der eingesparten Stromkosten und den Prozentsatz der ursprünglichen Stromkosten innerhalb einer Woche zu erhalten, wird die Produktionsplanung für die 44. Woche des Jahres 2023 mit dem Algorithmus optimiert. Dabei ist zu erkennen, dass die Einsparpotenziale jeden Tag unterschiedlich sind, weil der Strompreis nicht täglich konstant bleibt (siehe Abb. 6.3).

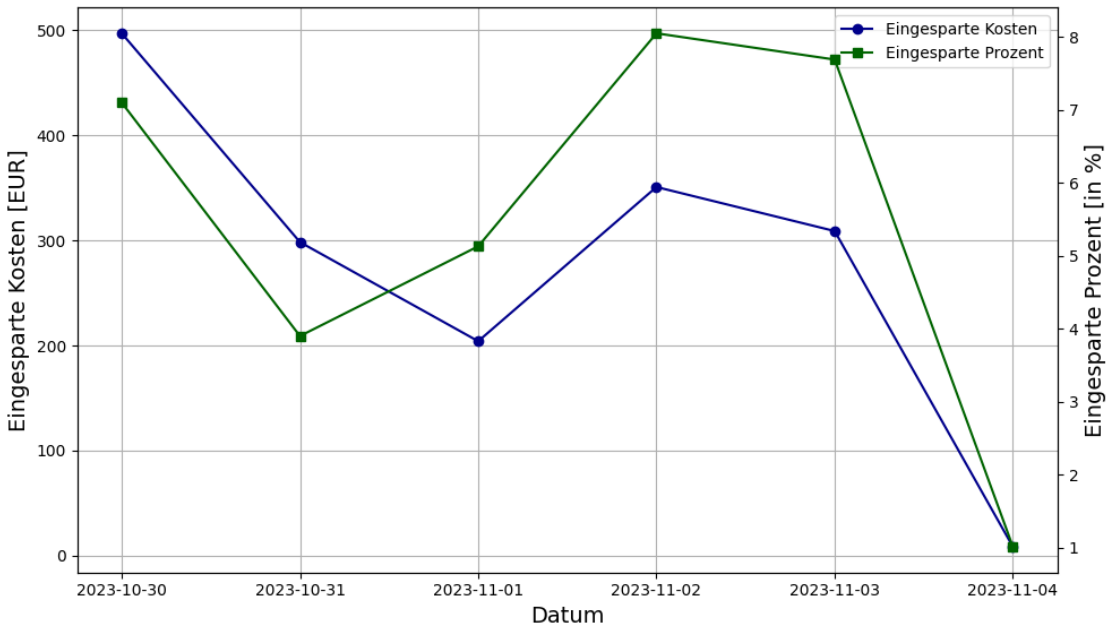

Abb. 6.3: Eingesparte Stromkosten und Prozentanteile der 44. Woche 2023. Der erste Tag (30.10.2023) ist ein Montag.

In Abb. 6.3 ist zu erkennen, dass in der 44. Woche des Jahres 2023 das höchste Einsparpotenzial bei 350 EUR liegt, was 8,05 % der ursprünglichen Gesamtkosten repräsentiert. Durch die Summierung aller Datenpunkte ergeben sich durchschnittlich eingesparte Stromkosten von 278,01 EUR und ein Prozentanteil von 5,48 %. Zusätzlich kann untersucht werden, ob es eine signifikante Änderung in der Korrelation zum Strompreis zwischen der nicht optimierten und der optimierten Produktionsplanung gibt (Verteilung der herzustellenden geplanten Kästen). Bevor die Produktionsplanung oder der Stromverbrauch optimiert werden, treten häufig hohe Produktionsaktivitäten in den Stunden mit den teuersten Strompreisen auf. Gleichzeitig sind die Produktionsaktivitäten in den Stunden mit den günstigsten Preisen niedriger. Dies kann zu einer hohen positiven Korrelation zwischen dem Strompreis und der geplanten Kastenanzahl führen. Im Gegensatz dazu sollte diese positive Korrelation nach der Optimierung der Produktionsplanung verringert oder in eine negative Korrelation umgewandelt werden. Die Spearman-Korrelation (siehe Formel 2.6) zwischen der nicht optimierten und der optimierten Produktionsplanung (Kastenanzahl) bezüglich des Strompreises wird in Tab. 6.1 dargestellt, um die Effektivität der Optimierung zu beurteilen.

Tab. 6.1: Spearman-Korrelationen zwischen optimierter und nicht optimierter Produktionsplanung und Strompreisen mit p-Werten

	Nicht optimierte Kastenanzahl	Optimierte Kastenanzahl
Strompreis	0,46 ($p < 0{,}001$)	0,16 ($p = 0{,}038$)

Tab. 6.1 zeigt, dass die Korrelation zwischen dem Strompreis und der nicht optimierten Produktionsplanung bei 0,46 liegt. Nach der Optimierung der Produktionsplanung ist dieser Wert auf 0,16 gesunken, was bedeutet, dass der Algorithmus die Verteilung der herzustellenden geplanten Kästen basierend auf dem Strompreis effektiver anpasst. Auf diese Weise wird vermieden, zu viele Kästen während der teuersten Stunden und zu wenige während der günstigsten Stunden herzustellen.

Die Korrelation wurde durch den Optimierungsalgorithmus nicht wie angenommen in eine negative umgewandelt. Der mögliche Grund dafür ist, dass es in der Realität nicht immer umsetzbar ist, die Produktion der geplanten Kästen auf kostengünstigere Zeiten zu verschieben. Dies liegt daran, dass die Produktionskapazität an manchen Tagen mit günstigen Phasen wegen technologischer Beschränkungen oder begrenzter Ressourcen bereits voll ausgelastet ist. Aus diesem Grund ist es nicht immer möglich, einige der geplanten Kästen von Spitzenzeiten auf günstigere Stunden zu verschieben.

7 Fazit und Ausblick

Durch die Anwendung der Modelle ARIMA, LSTM und XGBoost wurden die Stromschwankungen der Formanlage bei dem Unternehmen Fondium modelliert und der künftige Energieverbrauch prognostiziert. Vor der Modellierung wurden die Rohdaten durch Feature Engineering verarbeitet, wodurch die Leistung der drei Modelle maximiert und die benötigte Trainingszeit minimiert wurde. Nach Abschluss der Modellierung wurden diese Modelle anhand der Metriken MAE, RMSE und R2 sowie der benötigten Trainingsdauer durch Validierungs- und Testsätze evaluiert, um eine optimale Lösung für das Energiemanagement in der Produktion zu finden. Zusätzlich ermöglicht der Einsatz in der Produktionsplanung, dass die voraussichtlichen Stromkosten basierend auf dem prognostizierten Stromverbrauch kalkuliert werden. So kann die Menge der geplanten Produkte zwischen teuren und günstigen Stunden angepasst werden.

Mithilfe des Feature Engineerings wurde festgestellt, dass die Kastenanzahl besser als das Gießgewicht und das durchschnittliche Gießgewicht pro Kasten geeignet ist, um die allgemeinen Schwankungen im Stromverbrauch zu erfassen. Dabei kann die Modellierung der täglichen Stromschwankung durch Berücksichtigung von Arbeitstagen und Produktionsausfällen aus dem Produktionskalender weiter verbessert werden. Um den stündlichen Strombedarf abzubilden, können anstelle von 5-Minuten-Intervallen Stunden innerhalb eines Arbeitstages für eine effektive Feinjustierung genutzt werden.

Das ARIMA-Modell zeigte sich als das am wenigsten effektive Modell im Hinblick auf die Vorhersagegenauigkeit und die Trainingsdauer. Trotz der guten Leistung von ARIMA in vielen Anwendungsfällen, insbesondere bei stark saisonalen und stabilen Trends, war es in dieser speziellen Anwendung mit häufigen nicht-linearen Schwankungen und plötzlichen Änderungen nicht erfolgreich. Die längste Trainingszeit unter den drei Modellen und die geringste Vorhersagegenauigkeit limitieren seine Praktikabilität für den Einsatz in dynamischen und kostenkritischen Anwendungsbereichen wie der industriellen Energieverwaltung.

Im Vergleich dazu bot das LSTM-Modell eine bessere Leistung bei der Vorhersagegenauigkeit. Mit dem Konzept der Mustererkennung in sequenziellen Daten eignet sich LSTM daher grundsätzlich gut für Zeitreihenanalysen wie die Stromverbrauchsprognose. Durch seine Fähigkeit, langfristige Abhängigkeiten in den Daten zu lernen, lieferte es in dieser Arbeit präzise Vorhersagen. Dies macht LSTM zu einer starken Option für Anwendungen, bei denen Genauigkeit höchste Priorität hat. Jedoch hatte das LSTM-Modell eine längere Trainingsdauer, was in Situationen mit begrenzter Trainingszeit und Rechenkapazität ein Nachteil sein könnte.

Durch die Vielzahl an kombinierten Entscheidungsbäumen kann XGBoost komplexe nicht-lineare Beziehungen und Interaktionen in den Zeitreihendaten effizient abbilden. Dabei zeigte XGBoost ebenfalls eine hervorragende Leistung sowohl bei der Vorhersagegenauigkeit als auch bei der Trainingszeit. In dieser Arbeit bestätigte sich diese Fähigkeit, indem XGBoost ebenso präzise Vorhersagen wie LSTM lieferte und gleichzeitig die kürzeste Trainingszeit benötigte. Diese Effizienz qualifiziert es besonders für Situationen, in denen schnelle Entscheidungen auf Grundlage von aktuellen Daten notwendig sind.

Die Analyse des Stromverbrauchs eröffnet Einblicke in die potentiellen Energiekosten des Produktionsprozesses. In dieser Arbeit wurde zudem untersucht, inwiefern die optimierte Produktionsplanung durch präzise Prognose die Stromkosten beeinflussen kann. In diesem Zusammenhang wurde versucht, unter Beibehaltung des täglichen Produktionsvolumens eine Neuverteilung der Fertigungsaktivitäten vorzunehmen. Die Produktionsplanung wurde so angepasst, dass die Produktion in Zeiten hoher Strompreise reduziert und in kostengünstigeren Perioden gesteigert wurde. Eine Verbesserung der Produktionsplanung für die 44. Woche des Jahres 2023 wurde mithilfe der Nutzung des XGBoost-Modells simuliert. Dabei zeigt das Ergebnis, dass durchschnittlich 5,48 % der ursprünglichen Gesamtkosten in dieser Woche eingespart werden könnten. Da dies eine theoretische Berechnung ist, wurde die praktische Umsetzbarkeit im Unternehmen noch nicht berücksichtigt.

Allerdings sind Verbesserungen und Erweiterungen für die verwendeten Methoden und die implementierten Modelle möglich. Zum einen kann die in Kapitel 3.3 beschriebene Methode zur Identifizierung von Ausreißern verbessert werden, indem ML-Cluster-Verfahren wie Density-Based Spatial Clustering of Applications with Noise (DBSCAN) oder k-means verwendet werden, anstatt die Stromverbrauchswerte über 325 auf Basis der Histogrammanalyse pauschal als fehlerhaft eingetragene extreme Werte zu behandeln. Des Weiteren können die konkreten Feiertage in der Produktion besser berücksichtigt werden. Die Analyse zeigt, dass die Firma Fondium an manchen Feiertagen trotzdem Produkte herstellt, was das Anlernen des Vorhersagemodells verzerren kann.

Das verwendete DL-Modell LSTM, welches in Kapitel 4.2 beschrieben wird, kann auch mit der Technologie Compute Unified Device Architecture (CUDA) des Unternehmens Nvidia trainiert werden, was die Trainingsdauer möglicherweise noch weiter verkürzen würde. Zusätzlich gibt es mehrere Hyperparameter des XGBoost-Modells, wie zum Beispiel die maximale Baumtiefe und die Mindestanzahl der Beobachtungen pro Blatt, die potenziell die Leistung des XGBoost erhöhen könnten. Das Optimierungsmodell für die Produktionskosten verwendet den prognostizierten Stromverbrauch sowie den Strompreis aus einem anderen Vorhersagemodell der Webseite Energie-Charts. Dieses Modell könnte man in zukünftigen Arbeiten auch selbst entwickeln, um möglicherweise präzisere Strompreise und unabhängige Informationen zu erhalten.

Im Gegensatz zu herkömmlichen Datenanalysen bietet die Architektur der implementierten Modelle, die individuell anpassbare Parameter und Module nutzt, die Möglichkeit zur flexiblen Erweiterung. Zusätzliche Funktionen können integriert und weitere Analysewerkzeuge eingebettet werden, um die Modelle ARIMA, LSTM und XGBoost optimal auf die spezifischen Anforderungen der Stromverbrauchsprognose in industriellen Formanlagen anzupassen.

Literaturverzeichnis

[1] D. Kibira, Q. Hatim und G. Shao, „Integrating data analytics and simulation methods to support manufacturing decision making," in *2015 winter simulation conference (WSC)*, Huntington Beach, 2015, S. 2100-2111.

[2] M. H. G. Kraft, E. Frank und A. Podleisek, *Carbon Management in KMU: Strategien und Ansätze zur Klimaneutralität*, Wiesbaden: Springer Gabler, 2023.

[3] G. Box, G. Jenkins und G. Reinsel, *Time Series Analysis: Forecasting and Control*, 3. Aufl. New Jersey: Prentice Hall, 1994.

[4] S. Makridakis und M. Hibon, „ARMA models and the Box–Jenkins methodology," *Journal of Forecasting,* Bd. 3, Nr. 16, S. 147-163, 1997.

[5] Z. C. Toth, „Datenarchitektur," in *Handbuch Data Science und KI: Mit Machine Learning und Datenanalyse Wert aus Daten generieren*, 2. Aufl. München: Carl Hanser Verlag, 2022, S. 75-111.

[6] D. L. Olson und D. Wu, *Predictive Data Mining Models*, 2. Aufl. Singapore: Springer Nature Singapore, 2020.

[7] F. M. Salem, *Recurrent Neural Networks: From Simple to Gated Architecture*, Cham: Springer Nature Switzerland, 2022.

[8] Z. C. Lipton, J. Berkowitz und C. Elkan, „A critical review of recurrent neural networks for sequence learning," arXiv preprint arXiv:1506.00019, 2015.

[9] Y. Wang, D. Gan, M. Sun, N. Zhang, Z. Lu, und C. Kang, „Probabilistic individual load forecasting using pinball loss guided LSTM," *Applied Energy*, Bd. 235, S. 10-20, 2019.

[10] T. Chen und C. Guestrin, „XGBoost: A Scalable Tree Boosting System," in *Proceeding of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, San Francisco, 2016, S. 785-794.

[11] Y. Jiang, G. Tong, H. Yin und N. Xiong, „A Pedestrian Detection Method Based on Genetic Algorithm for Optimize XGBoost Training Parameters," in *IEEE Access*, Bd. 7, S. 118310-118321, 2019.

[12] Y. Xia, C. Liu, Y. Li und N. Liu, „A boosted decision tree approach using Bayesian hyper-parameter optimization for credit scoring," *Expert Systems with Applications*, Bd. 78, S. 225-241, 2017.

[13] M. Hebing und M. Manhembué, *Data Science Management: Vom Ersten Konzept Bis Zur Governance Datengetriebener Organisationen*, Heidelberg: O'Reilly Verlag, 2024.

[14] K. Backhaus, B. Erichson, S. Gensler, R. Weiber und T. Weiber, *Multivariate Analysemethoden: Eine anwendungsorientierte Einführung*, 17. Aufl. Wiesbaden: Springer Gabler, 2023.

[15] B. Auer und H. Rottmann, *Statistik und Ökonometrie für Wirtschaftswissenschaftler: Eine anwendungsorientierte Einführung*, 2. Aufl. Wiesbaden: Gabler Verlag, 2011.

[16] M. Elsaraiti, G. Ali, H. Musbah, A. Merabet und T. Little, „Time Series Analysis of Electricity Consumption Forecasting Using ARIMA Model," in *2021 IEEE Green Technologies Conference (GreenTech)*, Denver, CO, USA, 2021, S. 259-262

[17] M. Trabs, M. Jirak, K. Krenz und M. Reiß, *Statistik und maschinelles Lernen: Eine mathematische Einführung in klassische und moderne Methoden*, Berlin: Springer Spektrum, 2021.

[18] I. Frost, *Konfidenzintervalle und Standardfehler-Balken: Das Konzept verstehen und Ergebnisse angemessen interpretieren*, Berlin: Springer Spektrum, 2023.

[19] D. A. Dickey und W. A. Fuller, „Likelihood ratio statistics for autoregressive time series with a unit root," *Econometrica: Journal of the Econometric Society*, Bd. 49, Nr. 4, S. 1057-1072, 1981.

[20] A. Jung, *Maschinelles Lernen: Die Grundlagen*, Singapore: Springer Nature Singapore, 2024.

[21] S. Arlot und A. Celisse, „A survey of cross-validation procedures for model selection," *Statistics Surveys*, Bd. 4, S. 40-79, 2010.

[22] C. Bergmeir und J. M. Benitez, „Forecaster performance evaluation with cross-validation and variants," in *Proc. 2011 11th International Conference on Intelligent Systems Design and Applications*, 2011, S. 849-854.

[23] C. J. Willmott, „Some comments on the evaluation of model performance," *Bulletin of the American Meteorological Society*, Bd. 63, Nr. 11, S. 1309-1313, 1982.

[24] D. Chicco, M. J. Warrens, und G. Jurman, „The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation," *PeerJ Computer Science*, Bd. 7, Artikel-Nr. e623, 2021. [Online]. Verfügbar unter: https://doi.org/10.7717/peerj-cs.623

[25] J. Unpingco, *Python for Probability, Statistics, and Machine Learning*, 2. Aufl. Cham: Springer Verlag, 2019.

[26] J. Vanderplas, *Data Science mit Python: das Handbuch für den Einsatz von IPython, Jupyter, NumPy, Pandas, Matplotlib, Scikit-Learn*, übers. K. Lorenzen, Frechen: Mitp Verlag, 2018.

[27] B. V. Vishwas und A. Patel, *Hands-on Time Series Analysis with Python: From Basics to Bleeding Edge Techniques*, Berkeley: Apress Verlag, 2020.

[28] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, u.a., „SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, Bd. 17, Nr. 3, S. 261-272, 2020.

[29] P. Mishra, *PyTorch Recipes: A Problem-Solution Approach to Build, Train and Deploy Neural Network Models*, Berkeley: Apress Verlag, 2023.

[30] C. Wade und K. Glynn, *Hands-On Gradient Boosting with XGBoost and Scikit-Learn: Perform Accessible Machine Learning and Extreme Gradient Boosting with Python*. Birmingham: Packt Publishing Ltd., 2020.

[31] A. Zheng und A. Casari, *Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists*, Sebastopol: O'Reilly Media, 2018.

[32] J. Hirschle, *Machine Learning für Zeitreihen: Einstieg in Regressions-, ARIMA- und Deep Learning-Verfahren mit Python*, München: Hanser, 2021.

[33] Energie-Charts, *Stromproduktion und Börsenstrompreise in Deutschland in Woche 44 2023*. [Online]. Verfügbar unter: https://www.energy-charts.info/charts/price_spot_market/chart.htm?l=de&c=DE&year=2023&interval=week&week=44 (Zugriff am: 05. Juli 2024).

Eidesstattliche Erklärung

Hiermit erkläre ich, dass ich diese Bachelorarbeit selbstständig und nur unter Benutzung der angegebenen Literatur und Hilfsmittel angefertigt habe und alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, als solche gekennzeichnet sind. Die Arbeit wurde noch nicht bei einer Prüfungsbehörde eingereicht oder veröffentlicht. Zusätzlich erkläre ich, dass ich Generative KI-Technologien ausschließlich zur Verbesserung des Codes und der Rechtschreibung eingesetzt habe. Dabei wurden keine sensiblen Daten eingegeben oder ganze Texte generiert. Der Einsatz dieser Technologien beschränkte sich strikt auf technische Unterstützung ohne Einfluss auf die inhaltliche Eigenleistung.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

31. Juli 2024	Chin-I Feng
Datum	Unterschrift

Anhang

A1 LSTM-Code

Der in diesem Abschnitt präsentierte Codeauszug illustriert die Implementierung eines LSTM-Netzwerks zur Vorhersage des Stromverbrauchs. Der vollständige Quellcode ist zur weiteren Einsicht und detaillierteren Analyse im HTWG GitLab-Repository verfügbar.

```
import torch
import torch.nn as nn

class LSTM(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        """
        :param input_size: Anzahl der Merkmale in der Eingabedaten.
        :param hidden_size: Anzahl der Features in den versteckten Zuständen des LSTM.
        :param output_size: Die Anzahl der Features im Ausgabe-Layer des Modells.
        """
        super(LSTM, self).__init__()
        self.hidden_size = hidden_size  # Hidden dimensions
        self.output_size = output_size  # Number of output Layer
        self.lstm1 = nn.LSTM(input_size, hidden_size, batch_first=True)  # First hidden layer
        self.lstm2 = nn.LSTM(hidden_size, hidden_size, batch_first=True)  # Second hidden layer
        self.fc = nn.Linear(hidden_size, output_size)  # Readout Layer

    def forward(self, x):
        """
        :param x: Eingabetensor der Form (batch_size, sequence_length, input_size)
        :return: Tensor der Vorhersagen mit der Form (batch_size, output_size)
        """
        # Passing through the first hidden layer
        h0_1 = torch.zeros(self.output_size, x.size(0), self.hidden_size)
        c0_1 = torch.zeros(self.output_size, x.size(0), self.hidden_size)
        out, _ = self.lstm1(x, (h0_1, c0_1))

        # Passing through the second hidden layer
        h0_2 = torch.zeros(self.output_size, x.size(0), self.hidden_size)
        c0_2 = torch.zeros(self.output_size, x.size(0), self.hidden_size)
        out, _ = self.lstm2(out, (h0_2, c0_2))
        out = self.fc(out)
        return out
```

Abb. A.1: Implementierung des LSTM-Netzwerks

```python
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.001):
        """
        :param patience: Anzahl von Epochen ohne Verbesserung nach denen das Training gestoppt wird.
        :param min_delta: Minimale Veränderung des Verlustes, die als Verbesserung zählt.
        """
        self.patience = patience
        self.min_delta = min_delta
        self.best_loss = None
        self.epochs_no_improve = 0

    def reset(self):
        """
        Setzt die internen Zustände des EarlyStopping-Objekts zurück.
        """
        self.best_loss = None
        self.epochs_no_improve = 0

    def __call__(self, validation_loss):
        if self.best_loss is None or (self.best_loss - validation_loss > self.min_delta):
            self.best_loss = validation_loss
            self.epochs_no_improve = 0
        else:
            self.epochs_no_improve += 1

        if self.epochs_no_improve >= self.patience:
            print("Early stopping triggered")
            return True
        return False
```

Abb. A.2: Implementierung der Funktion EarlyStopping

```python
import numpy as np
import torch
import torch.nn as nn
from modules.lstm_utils import LSTM, train_lstm_model
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import TimeSeriesSplit

# Skalierung
scaler = MinMaxScaler(feature_range=(-1, 1))
for col in FEATURES + TARGET:
    df[col] = scaler.fit_transform(df[col].values.reshape(-1, 1))

# Umformung in ein Numpy-Array und Aufteilung in Trainings- und Testdaten
data_feat, data_target = [], []
data_feat = np.array(df[FEATURES]).reshape(-1, lookback, num_input)
data_target = np.array(df[TARGET]).reshape(-1, lookback, num_output)
split_index = int(len(data_feat) * 0.9)
train_feat = data_feat[:split_index]
train_target = data_target[:split_index]
test_feat = data_feat[split_index:]
test_target = data_target[split_index:]

# LSTM Netzwerke Initialisierung
lstm_model = LSTM(4, 16, 1)
loss_function = nn.MSELoss()
optimizer = torch.optim.Adam(lstm_model.parameters(), lr=0.07)
num_batch = 32

# Cross-Validation (cv)
for train_index, val_index in TimeSeriesSplit(n_splits=10).split(train_feat):
    X_train, X_val = train_feat[train_index], train_feat[val_index]
    y_train, y_val = train_target[train_index], train_target[val_index]

    # Modelltraining
    train_lstm_model(X_train, y_train, X_val, y_val,lstm_model, loss_function, optimizer, num_batch)

# Vorhersagen zu Testdaten machen
with torch.no_grad():
    X_test_torch = torch.from_numpy(test_feat).float()  # Umwandlung in PyTorch-Tensor
    output = lstm_model(X_test_torch)
    y_test_pred = output.numpy()
```

Abb. A.3: Codeausschnitte zur Implementierung eines LSTM-Modells

A2 Code zur Optimierung der Kastenanzahl

Der in diesem Abschnitt präsentierte Codeauszug und das Flussdiagramm illustrieren die Algorithmen zur Reduzierung der Kästen in teuren Stunden und zur Verteilung in kostengünstigeren Zeiten, um die Gesamtkosten zu minimieren.

```python
def adjust_kasten_anzahl(original_df):
    """
    :param original_df: Ein DataFrame, das die Kasten_Anzahl und Preise enthält.
    :return: Das angepasste DataFrame, in dem die Kasten_Anzahl optimiert wurde.
    """
    min_kasten_anzahl = 50
    max_kasten_anzahl = 170
    num_top_prices = 10
    num_bottom_prices = 14

    while True:
        df = original_df.copy()
        initial_total = df['Kasten_Anzahl'].sum()
        top_prices = df.nlargest(num_top_prices, 'Preis')
        bottom_prices = df.nsmallest(num_bottom_prices, 'Preis').drop(top_prices.index, errors='ignore')

        reduced_quantity = 0
        for idx in top_prices.index:
            if df.loc[idx, 'Kasten_Anzahl'] > min_kasten_anzahl:
                reduced_quantity += df.loc[idx, 'Kasten_Anzahl'] - min_kasten_anzahl
                df.loc[idx, 'Kasten_Anzahl'] = min_kasten_anzahl  # Setze die Anzahl auf das Minimum

        for idx in bottom_prices.index:
            max_add = max_kasten_anzahl - df.loc[idx, 'Kasten_Anzahl']
            if max_add > 0:
                add_quantity = min(reduced_quantity, max_add)
                df.loc[idx, 'Kasten_Anzahl'] += add_quantity
                reduced_quantity -= add_quantity

        current_total = df['Kasten_Anzahl'].sum()
        if current_total == initial_total:
            original_df.update(df)
            break
        else:
            num_top_prices -= 1
            if num_top_prices < 1:
                break
    return original_df
```

Abb. A.4: Implementierung der Kastenanzahl-Optimierung

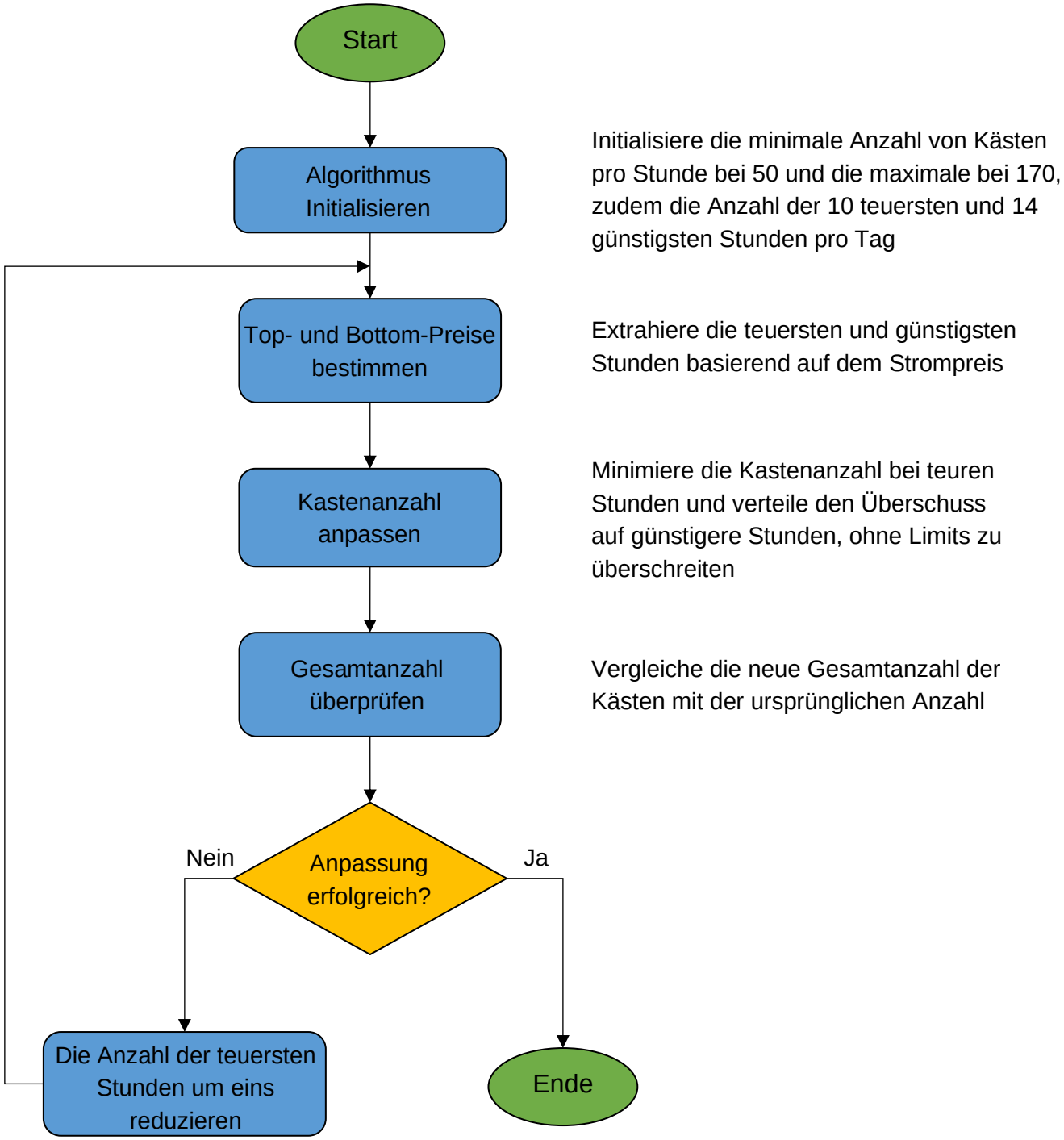

Abb. A.5: Flussdiagramm der Kastenanzahl-Optimierung. Der Input besteht aus dem Strompreis P(t) und der nicht optimierten Kastenanzahl n(t), während der Output die optimierte Kastenanzahl n_opt(t) ist.